

Iterative Planning for Multi-Agent Systems: An Application in Energy-Aware UAV-UGV Cooperative Task Site Assignments

Neelanga Thelasingha¹, A. Agung Julius², *Senior Member, IEEE*, James Humann³,
Jean-Paul Reddinger, James Dotterweich, and Marshal Childers

Abstract—This paper presents an iterative planning framework for multi-agent systems with hybrid state spaces. The framework uses transition systems to mathematically represent planning tasks and employs multiple solvers to iteratively improve the plan until computational resources are exhausted. When integrating different solvers for iterative planning, we establish theoretical guarantees for recursive feasibility. The proposed framework enables continual improvement of solutions to reduce sub-optimality, efficiently using allocated computational resources. The proposed method is validated by applying it to an energy-aware UAV-UGV cooperative task site assignment problem. The results demonstrate continual solution improvement while preserving real-time implementation ability compared to algorithms proposed in the literature.

Note to Practitioners—This paper presents an iterative planning solution for cooperative planning problems in multi-agent systems, which integrates multiple solvers to create an optimization framework. The proposed planning framework has been theoretically validated and applied in an energy-aware cooperative planning scenario for multi-vehicle task site assignments. The proposed framework can be applied to plan for any generalized task site assignment using multiple solvers iteratively.

Index Terms—Iterative planning, transition systems, multi-agent planning, energy-aware routing.

I. INTRODUCTION

RECENT advances in robotics and control have enabled multi-agent systems that operate in complex real-world environments. Operating such systems often involves solving complex planning problems in multi-dimensional state spaces. Real-world implementations of such solutions have

wide applications, such as in Unmanned Aerial/Ground Vehicle (UAV/UGV) systems [1], [2], underwater vehicles [3], route planning for a multi-robot station in a manufacturing cell [4], station inventory management systems [5] and automated material handling systems [6]. Many of these applications involve planning trajectories in high-dimensional state spaces. Among them, route planning tasks for UAV-UGV teams [7], [8], [9], [10], [11] involve specific challenges due to the heterogeneous dynamics. Further, agents' operations are limited by several constraints, including energy usage [12], [13] and the assigned task, for example, package delivery [14].

A. Motivation

In general, most cooperative planning tasks can be formulated as a variant of the well-known NP-hard vehicle routing problem [15], [16] and its two-echelon vehicle routing problem counterpart [17], [18], but the complexity increases when energy dynamics must also be considered [8], [19]. The planned routes must be feasible under the energy availability constraint for each vehicle. When energy recharging is considered, additional constraints are imposed [20]. In any solution, each agent must operate within the bounds of the energy capacity and follow the defined discharge/recharge dynamics [21], [22]. Therefore, including the energy dimension in energy-aware planning tasks further increases the complexity [6], [9]. The state spaces in such problems are hybrid, with multiple continuous and discrete dimensions. When the applications include persistent or periodic delivery tasks that require receding horizon planning, such as the cases in [4], [11], and [23], the allowed computation resources per planning step are bounded.

In the literature, many approaches include discretizing the state space to speed up computation. These formulations enable the application of Traveling Salesperson Problem (TSP) solvers and Vehicle Routing Problem (VRP) solvers in the discretized state space [9], [16]. Further, a discrete formulation of the problem allows the energy dynamics and task site assignment to be encoded as integer and logical constraints. Then, the cooperative route planning task can be solved as a Mixed-Integer Program (MIP) [24]. However, solving the MIP for optimality is computationally expensive. Therefore,

Manuscript received 8 December 2023; revised 19 March 2024; accepted 4 May 2024. Date of publication 14 May 2024; date of current version 14 February 2025. This article was recommended for publication by Associate Editor H.-J. Kim and Editor J. Yi upon evaluation of the reviewers' comments. This work was supported by the DEVCOM Army Research Laboratory under the Cooperative Research Agreement under Grant W911NF-21-2-0283 79054-CI-ARL. (Corresponding author: Neelanga Thelasingha.)

Neelanga Thelasingha and A. Agung Julius are with the Center for Mobility with Vertical Lift (MOVE), Rensselaer Polytechnic Institute, 2303 Jonsson Engineering Center, Troy, NY 12180 USA (e-mail: thelan@rpi.edu; agung@ecse.rpi.edu).

James Humann, Jean-Paul Reddinger, James Dotterweich, and Marshal Childers are with the DEVCOM Army Research Laboratory, Aberdeen, MD 21005 USA (e-mail: james.d.humann.civ@army.mil; jean-paul.f.reddinger.civ@army.mil; james.m.dotterweich.civ@army.mil; marshal.a.childers.civ@army.mil).

Digital Object Identifier 10.1109/TASE.2024.3398663

1558-3783 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

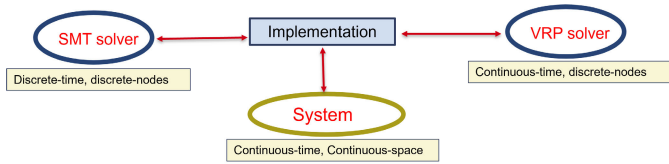


Fig. 1. Example - SMT solver that operates in discrete-time on discrete-nodes and a VRP solver that operates in continuous-time on discrete-nodes generating implementations for a system in continuous-time on continuous-space.

hierarchical multi-level optimization approaches have been proposed to reformulate the constraints or break the problem into smaller subproblems and solve them [8], [25]. This still requires solving an NP-hard optimization problem, which scales up very quickly with the number of agents and task sites. Another approach is data-driven methods that leverage computation and training time to learn a policy that can be deployed [26], [27]. However, these methods suffer from sample inefficiency and lack of safety guarantees [28].

We identify the need for a solver for the task site assignment problem with anytime computational property. This allows a solver to provide feasible but possibly suboptimal solutions at any point during its execution. To this end, we propose a unifying iterative planning framework for multi-agent systems. Our framework first finds a feasible solution using a constraint solver and iteratively optimizes the solution, given the availability of computation resources. See the example of its implementation in Figure. 1, where a Satisfiability Modulo Theory (SMT) solver provides a feasible solution, and successive solvers improve upon the previous solutions. The challenge here is that these subsequent solvers operate in varying hybrid state spaces. Therefore, a unifying framework where consensus can be achieved by all solvers is essential. In this paper, we proposed to use the concept of transition systems as the basis for this unifying framework [29], [30].

B. Overview

Transition systems have been widely used as a theoretical formulation to represent problems in complex systems with state spaces with continuous and discrete dimensions [29]. Standard problem discretization procedures can be applied to formulate sampled transition systems and enable faster solution computation. Our proposed framework uses multiple solvers that are called iteratively to improve the existing solution. An example approach is to call a faster solver initially to generate a feasible plan and then to iteratively call the next solvers to improve the plan until computational resources are exhausted. However, the solvers have different sampling and discretization schemes and therefore require a unifying framework to support integration. Therefore, we derive and prove the necessary mathematical conditions to guarantee the recursive feasibility, i.e., that all solvers agree on each other's solutions and identify them as feasible plans to satisfy the task site assignment.

We apply the proposed framework on an energy-aware UAV-UGV cooperative routing task. In this application, we formulate team-level and agent-level planning problems. The effectiveness of our framework is demonstrated through improving the objective function, i.e., the plan execution time,

when compared to standard optimization solvers like mixed integer programming solvers [7], [19] used in state-of-the-art solutions.

C. Contributions

The main contributions of this paper are listed and described as follows.

- 1) Iterative planning framework that integrates multiple solvers for planning in multi-agent systems. The iterative planning framework enables fast computation of a feasible plan and continual improvement of the plan given computation time. The framework also includes a shrinking horizon implementation that replans at each step of execution for further optimization over time.
- 2) Theoretical analysis of recursive feasibility of the iterative planning framework. Using transition systems, we introduce a mathematical framework to unify different solvers for planning in multi-agent systems with hybrid state spaces for any general task site assignment problems. We derive and prove the conditions to preserve the recursive feasibility of the proposed algorithms.
- 3) Application in energy-aware UAV-UGV cooperative task site assignments. We apply the proposed framework to solve energy-aware UAV-UGV cooperative task site assignments. We cast the problem in the proposed mathematical framework to prove the recursive feasibility for two solvers that are called iteratively to optimize the plan. The shrinking horizon execution enables the continual improvement of the plan by re-planning at each step.
- 4) Comparative result analysis of proposed approach against optimization solvers and state-of-the-art methods. We compare the compute time and total plan time of the proposed approach against standard MIP solver and state-of-the-art methods proposed in literature such as Bayesian optimization and evolutionary methods. We also include a sensitivity analysis of the proposed method against the complexity of the problem by varying the number of task sites and the number of agents.

II. RELATED WORK

In the literature, there are solutions for multi-agent planning and routing for specific tasks, such as package delivery and distribution assignments [1], [2], [31]. However, adapting these solutions to the generalized tasks site assignment, which includes specifications on required visits and some general tasks at the intended sites is challenging [1], [8], [9], [10]. Some literature considers agents that do not interact [1], [25]. Others consider agent-agent interactions, for example, between ground vehicles and aerial vehicles in package delivery scenarios [2], [17], [18], [32] or as a refueling or recharging resource in surveillance-type tasks [7], [8], [9], [11], [14] where cooperative tasks can be formulated as a two-echelon vehicle routing problem. Our paper captures agent-agent interaction in a generalized task site assignment.

Energy-aware planning problems [5], [6], [8], [9], [15], [16], [20], [33] arise when energy consideration is included in planning, adding further complexities by modeling the energy dynamics [24] and optimal planning for the refueling/recharging instances [22]. In this paper, we formulate the planning problem for cooperative tasks as an energy-aware planning problem. Usual energy-aware planning problem formulations include the VRP, which is a generalization of the well-known NP-hard traveling salesperson problem [10], [12], [14], [34], [35]. Adding more constraints such as health monitoring [11], time constraints [12], [23], [33], and possible adversarial attacks [34], [35] further exacerbates the problem complexity. In this work, we first focus on finding feasible solutions and iteratively optimize them later.

Once the planning problem is formulated, the solution method depends on the problem variable representation. Solving the problem optimally in the naturally continuous state space would yield optimal solutions [3], but with high computation complexity. On the other hand, a discretization [7] of the state space with graphical representations [14], [36] enables the use of VRP solvers and MIP solvers [8], [9]. In recent work, optimal solutions have been achieved by solving an MIP [7], [19]. However, this is not applicable for large problems [24], [34], [36]. Some papers, for example, [7], [12], [13], [36], use a heuristic-based approach that is challenging to generalize. There are some other techniques such as adaptive neighborhood search [32] that can improve a solution. However, obtaining the optimal solutions for a large number of heterogeneous interacting agents is not possible.

Another class of recent work proposes to use a probabilistic formulation of the planning problem [1], [11]. Solving such problems usually requires a data-driven learning-based approach [21], [37], which learns an optimal policy [3]. However, this approach usually requires a lot of samples and extensive training. Further, it is not possible to establish safety or performance guarantees with such approaches, unlike greedy methods [10].

Some published work uses a hierarchical approach, where an initial step of clustering of task sites simplifies the problem for the next step [8], [9], [19], [33], [38]. In the next step, [6], [7], [24], [34] use a MIP solver while [8], [9], [33] use Bayesian optimization. However, an optimization problem still needs to be solved to obtain a specification-satisfying solution.

In contrast with the previous approaches, our proposed approach uses an iterative framework, where, at the first step, a constraint solver generates a feasible solution. Then, given the availability of the computation resources, it is further optimized through iterative solvers. Here, it is important to note that at each step of the iterative framework, a feasible solution always exists at hand. This formulation requires a multi-level optimization framework [38] for multiple solvers with hybrid state spaces.

We use transition systems, commonly used in literature for modeling complex problems with hybrid state spaces [39], to define the framework and conduct verification. In [40], monotonic transition systems are used for symbolic reachability analysis, while [41] and [42] conduct verification in fuzzy automata. Further, [43] and [44] present simulation functions

for stochastic hybrid automata using transition systems. Their modeling capabilities have been applied for decision problems such as signaling in traffic networks in [45]. Further, in [46] epistemic transition systems are defined to represent coalitions and distributed knowledge. Therefore, transition systems present a well-defined framework with modeling capabilities to generate verifiable results in hybrid systems.

III. PRELIMINARIES

In this section, we introduce and elaborate on the required concepts and definitions used in this paper.

A. Notation

- S : Infinite state transition system.
- X : Set of states in the transition system S .
- T : Set of transitions in the transition system S .
- L : Set of labels associated with transitions in T .
- Y : Set of task sites in the transition system S .
- H : Output map of the transition system S .
- p_x^j : x-coordinate of the j^{th} vehicle.
- p_y^j : y-coordinate of the j^{th} vehicle.
- e_j : Energy level of the j^{th} vehicle.
- f_j : Label of the j^{th} vehicle.
- x_j : State of the j^{th} vehicle, with components $[p_x^j, p_y^j, e_j, f_j]$.
- x : System state.
- γ_d : Sampling interval for the time dimension.
- γ : Timestamp for the implementation of the trajectory.
- X_d : Sampled state space.
- Y_d : Sampled output space.
- T_d : Set of transitions in the sampled transition system S_d .
- τ : Execution trajectory in the original transition system S .
- M : Specification in the original transition system S .
- M_d : Discretized specification.
- Ξ : Task site assignment.
- $\Sigma(S)$: Set of all execution trajectories in S .
- $\Sigma^N(S)$: Set of all execution trajectories of length N in S .
- ζ_τ : Implementation of a trajectory τ .
- $\xi_\zeta(\gamma)$: Output at timestamp γ of the implementation ζ .
- N_y : Cardinality of the assignment for the task site.
- $\hat{Y}_i$: Output class in the task site assignment.
- $\hat{X}_i$: Key state class formed by output class $\hat{Y}_i$.
- $\hat{T}_j$: Key transition class formed by output class $\hat{Y}_i$.
- f_S : Objective function in transition system S .
- $t_{\text{compute}}^{\max}$: Maximum allowed compute time for the solver.
- $t_{\text{compute}}^\Omega$: Compute time spent by the solver Ω .
- $\text{Image}(\xi_\zeta)$: Image of the output behavior of ζ .
- τ_d : Trajectory in the sampled transition system.
- $\zeta_\Sigma(S)$: Set of all feasible implementations in the system S .

B. Transition Systems

Many real-world planning tasks involve complex dynamical systems with finite and infinite state spaces along with their combinations. A proper mathematical formulation of these tasks and involved systems is essential when formulating planning strategies. Systems that are modeled as transition

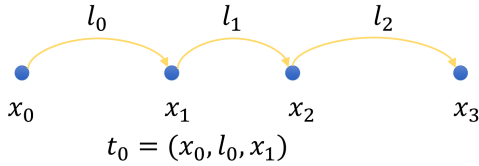


Fig. 2. States, labels and transitions.

systems offer a well-defined framework for describing planning tasks and strategies. These systems provide a formal framework that captures the discrete changes and interactions within a system's state space over time. By representing state transitions and the corresponding actions or events, transition systems offer a concise and abstract representation of the underlying dynamics.

The advantages of using transition systems lie in their ability to handle both finite and infinite state spaces, accommodating a wide range of real-world scenarios. They enable the formulation of planning problems by defining the set of possible states, actions, and transitions between states. This clarity in representation facilitates the development of planning strategies, allowing systematic exploration of various solvers or solving strategies. However, alternative formalisms such as differential equations or probabilistic models might be preferred in cases where continuous dynamics play a crucial role or uncertainty is inherent.

Throughout this paper, we use the idea of transition systems as the mathematical model that describes a dynamic phenomenon [30]. This enables us to model systems that evolve in finite state spaces with discrete transitions as well as systems with infinite and continuous state spaces that are described by differential equations. We formally define a transition system as follows.

Definition 1 (Transition System): A transition system is defined as a quintuple $S = (X, T, L, Y, H)$, where X is the state space (or set of states), L is the set of transition labels, and $T \subset X \times L \times X$ is the set of transitions, where a transition

$$t = (x, \ell, x')$$

denotes a transition from a state $x \in X$ to the state $x' \in X$, which is labeled by the symbol $\ell \in L$. Here, ℓ denotes the time duration for the transition t as shown in Figure 2. The set of outputs is denoted by Y and $H: X \rightarrow Y$ is the output map.

Notation 1: For a transition system $S = (X, T, L, Y, H)$, the fact that $(x, \ell, x') \in T$ is also denoted by $x \xrightarrow{\ell} x'$ or $x \xrightarrow{\ell} x'$ if the set of transitions T is clear from the context.

Notation 2: The set of all strings (i.e., finite sequences) of symbols in L is denoted by L^* .

C. Trajectories and Implementations

We define the evolution of a dynamic phenomenon within a transition system through sequential transitions as an execution trajectory.

Definition 2 (Execution Trajectories): For any positive integer N , a sequence $\tau := x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$ where $x_0, \dots, x_N \in X$ and $\ell_0, \dots, \ell_{N-1} \in L$ is said to

be an execution trajectory of $S = (X, T, L, Y, H)$ iff transition $t_k = (x_k, \ell_k, x_{k+1})$ exists in S , i.e., $t_k \in T$ for all $k \in \{0, \dots, N-1\}$. The length of the execution trajectory τ is defined as N .

Notation 3: For a transition system $S = (X, T, L, Y, H)$, the set of all execution trajectories of S is denoted by $\Sigma(S)$. The set of all execution trajectories of length N of S is denoted by $\Sigma^N(S)$.

Notation 4: For a transition system $S = (X, T, L, Y, H)$, any state $\hat{x} \in X$ is said to be in an execution trajectory $\tau := x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$ if $\hat{x} = x_i$ for some $i \in \{0, \dots, N\}$ and it is denoted by $\hat{x} \in \tau$.

Notation 5: For a transition system $S = (X, T, L, Y, H)$, any transition $t \in T$ is said to be in an execution trajectory $\tau := x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$, if $t = (x_i, \ell_i, x_{i+1})$ for some $i \in \{0, \dots, N-1\}$ and it is denoted by $t \in \tau$.

Next, we define implementations of trajectories. An implementation describes the system state's behavior when a trajectory is executed in real-time, following the transitions. Note that we associate the labels of transitions with the time duration of that transition execution.

Definition 3 (Implementation): Let a transition system be S and an execution trajectory be $\tau = x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$. Then an implementation of the trajectory τ is defined as a function of time $\zeta_\tau: (0, \sum_{i=0}^{N-1} \ell_i) \rightarrow X$ such that, for all $i \in \{0, 1, \dots, N\}$, $\zeta_\tau(\sum_{k=0}^{i-1} \ell_k) = x_i$. We denote the set of implementations of all trajectories of S as $\zeta_{\Sigma(S)}$.

D. Monotone Transition Systems

Here, we introduce an ordering relation $\geq$ on the states of a transition system. This allows comparison and ordering of the states according to some goodness measure. For example, let the state $x \triangleq [x_p, x_f]$ where $x_p \in \mathbb{R}^2$ denotes the position of an agent and $x_f \in \mathbb{R}$ represents the fuel level. Let us define the fuel level as the goodness measure. Thus, given a state $x = [x_p, x_f] \in X$, any state $x' = [x_p, x'_f] \in X$ is greater or equal in the partial ordering if $x'_f \geq x_f$ and it is denoted as $x' \geq x$. We assume that $\geq$ is *reflexive*, i.e., $x \geq x$ for any $x \in X$. Next, we define the monotone properties of transition systems in terms of partial ordering.

Definition 4 (Monotone Transition Systems): Given a transition system $S = (X, T, L, Y, H)$ and a partial order relation $\geq$ on X , S is said to be monotone with respect to $\geq$ iff the following statement is true for every $x_1, x'_1 \in X$ and $\ell \in L$, such that $(x_1, \ell, x'_1) \in T$: For all $x_2 \in X$ such that $x_2 \geq x_1$, there exists a transition $(x_2, \ell, x'_2) \in T$.

An implication of a transition system $S = (X, T, L, Y, H)$, being monotone with respect to $\geq$ is that if a sequence $x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$ is known to be an execution trajectory of S , then for any initial state $\hat{x}_0$ such that $\hat{x}_0 \geq x_0$, there is an execution trajectory $\hat{x}_0 \ell_0 \hat{x}_1 \ell_1 \cdots \ell_{N-1} \hat{x}_N$ where $\hat{x}_1 \geq x_1, \dots, \hat{x}_N \geq x_N$.

Therefore, in a monotone transition system, for a trajectory $\tau = x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$, replacement of a state $x_i \in \tau$ by a state $\hat{x}_i$ such that $\hat{x}_i \geq x_i$, results in a trajectory $\hat{\tau}$ with $\hat{x}_j \geq x_j$ for all j such that $j > i$.

Notation 6: For a transition system $S = (X, T, L, Y, H)$, that is monotone with respect to $\geq$, if $\tau = x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$ is known to be a trajectory of S , then for any initial state $\hat{x}_0 \geq x_0$, there is an execution trajectory, $\hat{\tau} = \hat{x}_0 \ell_0 \hat{x}_1 \ell_1 \cdots \ell_{N-1} \hat{x}_N$ where $\hat{x}_1 \geq x_1, \dots, \hat{x}_N \geq x_N$, which is denoted by $\hat{\tau} \succeq_{\tau} \tau$. Here, $\succeq_{\tau}$ denotes the partial ordering notation lifted to the set of trajectories.

E. Output Behavior

While an execution trajectory exists in the state space of a system, observations can be made in the output space. Similarly, for any implementation that returns the system state at any given time, observations can be made in the output space by passing the returned state through the output map. The corresponding output sequence of an implementation observed through the output map H is defined as the Output Behavior of an implementation.

Definition 5 (Output Behavior): For a given execution trajectory $\tau = x_0 \ell_0 x_1 \ell_1 \cdots \ell_{N-1} x_N$ of a transition system $S = (X, T, L, Y, H)$, and its implementation ζ , a function of time $\xi_{\zeta}: (0, \sum_{i=0}^{N-1} \ell_i) \rightarrow Y$ where,

$$\xi_{\zeta}(\gamma) = H(\zeta(\gamma)), \quad \forall \gamma \in (0, \sum_{i=0}^{N-1} \ell_i), \quad (1)$$

is defined as the output behavior of an implementation ζ .

F. Task Site Assignment in a Transition System

The goal of a task site assignment is to observe a specific set of outputs in the output space of a transition system.

Definition 6 (Task Site Assignment): For a transition system $S = (X, T, L, Y, H)$, a task site assignment is a collection of output classes, $\Xi = \{\hat{Y}_1, \hat{Y}_2, \dots, \hat{Y}_{N_y}\}$, where each $\hat{Y}_i \subset Y$. Here N_y is the cardinality of the assignment. If there exists an implementation ζ and its output behavior ξ_{ζ} such that $\hat{Y}_i \cap \text{Image}(\xi_{\zeta}) \neq \emptyset$ for all $i = 1, 2, \dots, N_y$, then ξ_{ζ} is said to satisfy the task site assignment Ξ_N .

A planning problem is formulated as finding an execution trajectory in the transition system that produces the given outputs of the task site assignment when implemented.

G. Sampled Transition System

We propose a solution framework that can be employed to generate feasible yet sub-optimal solutions faster and optimize them given the computation resources and time. To this end, we discretize the infinite state system in focus through a sampling procedure as shown in Figure 3. This involves a redefinition of the system as a sampled transition system.

Definition 7 (Sampled Transition System): Given an infinite state transition system, $S = (X, T, L, Y, H)$, we define a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ by sampling each dimension of the state such that the sampled state space $X_d \subset X$. The set of transitions is denoted as $T_d = (X_d \times L \times X_d) \cap T$. Further, the set of outputs $Y_d = \{y \in Y \mid \exists x_d \in X_d \text{ s.t. } y = H(x_d)\}$.

The sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ is now fully defined. Next, we define the specifications in

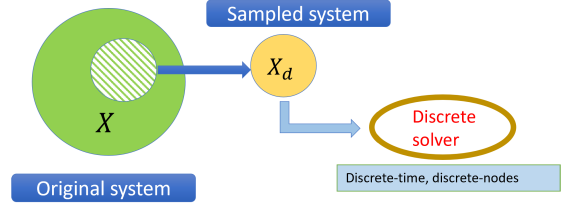


Fig. 3. Sampled transition system for a discrete solver.

the sampled transition systems, their key states, and key transitions.

H. Key States, Key Transitions, and Specifications

For a given task site assignment in the output space of the infinite system, we can find sets of states in a sampled system that would be mapped to the task site assignments. We identify these sets of states as key state classes.

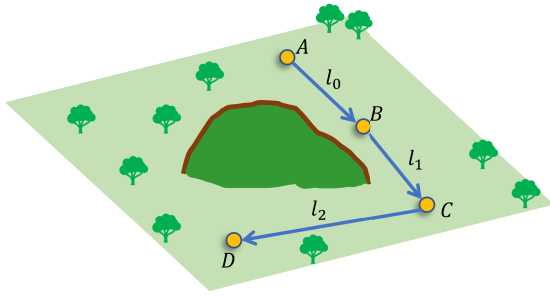
Definition 8 (Key State Classes): Let $S_d = (X_d, T_d, L, Y_d, H)$ be a sampled transition system of an infinite state transition system, $S = (X, T, L, Y, H)$. For a given task site assignment $\Xi = \{\hat{Y}_1, \hat{Y}_2, \dots, \hat{Y}_{N_y}\}$, each $\hat{Y}_i \in \Xi$ such that $\hat{Y}_i \cap Y_d \neq \emptyset$ is said to form a key state class $\hat{X}_i$ for Ξ in the sampled transition system S_d , i.e., $\hat{X}_i \subset X_d$, where $\hat{X}_i = \{x \in X_d \mid \exists y \in \hat{Y}_i \text{ s.t. } y = H(x)\}$.

For a given task site assignment in the output space of the infinite system, it is possible that outputs in some $\hat{Y}_i$ do not exist in the output space of the sampled system, i.e., $\hat{Y}_i \cap Y_d = \emptyset$. Intuitively, this is because some key outputs happen in between samples. For such key outputs, we define key transition classes in the sampled transition system. Figure 4 illustrates an example instance, where Figure 4a shows a system that contains all key states and Figure 4b shows a sampled system that does not contain all key states, thus represents the missing states as key transitions.

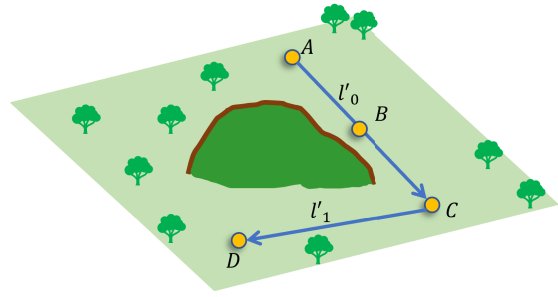
Definition 9 (Key Transition Classes): Let $S_d = (X_d, T_d, L, Y_d, H)$ be a sampled transition system of an infinite state transition system, $S = (X, T, L, Y, H)$. For a given task site assignment $\Xi = \{\hat{Y}_1, \hat{Y}_2, \dots, \hat{Y}_{N_y}\}$, each $\hat{Y}_i \in \Xi$ such that $\hat{Y}_i \cap Y_d = \emptyset$ is said to form a key transition class $\hat{T}_i$ for Ξ in the sampled transition system S_d , such that $\hat{T}_i \subset T_d$. Any transition $\hat{t} = (x_1, \ell, x_2) \in T_d$ with a label $\ell \in L$ and $x_1, x_2 \in X_d$ belongs to key transition class i , i.e., $\hat{t} \in \hat{T}_i$, if there exists an implementation ζ_{τ} for a trajectory $\tau = x_1, \ell, x_2$ and a timestamp $\gamma \in (0, \ell)$ such that $H(\zeta_{\tau}(\gamma)) \in \hat{Y}_i$.

Now, we have defined key transition classes that arise from the existence of a certain implementation whose output behavior contains task sites that do not exist in the output space of the sampled transition system. Next, we define the specification as the set of trajectories in the transition system that contains states from key state classes and transitions from key transition classes.

Definition 10 (Specification): Let S_d be a sampled transition system, $\hat{X}_i$ for $i = 0, \dots, N_x$ be a set of key state classes, $\hat{T}_j$ for $j = 0, \dots, N_t$ be a set of key transition classes for a task site assignment Ξ . A specification M is a set of execution trajectories of S_d , i.e., $M \subset \Sigma(S)$, such that for each trajectory



(a) Transition system S with state space $X = \{A, B, C, D\}$. Key states are $\{A, B, C, D\}$ where trajectory $\tau = A, \ell_0, B, \ell_1, C, \ell_2, D$ contains them.



(b) Sampled transition system S_d with state space $X_d = \{A, C, D\}$. Key states are $\{A, C, D\}$ and key transition is (A, ℓ'_0, C) where trajectory $\tau_d = A, \ell'_0, C, \ell'_1, D$ contains them.

Fig. 4. Key states and Key transitions: The state space X of the transition system S contains all key states. However, sampled transition system S_d only contains A, C, D . Thus, key state B forms a key transition for τ_d as $t = (A, \ell'_0, C)$ as there exists an implementation for τ_d that traverses through B .

$\tau \in M$, the following holds: (1) For each key state class $\hat{X}_i$ for $i = 0, \dots, N_x$, there exists a state $x \in \hat{X}_i$ such that $x \in \tau$. (2) For each key transition class $\hat{T}_j$ for $j = 0, \dots, N_t$, there exists a transition $t \in \hat{T}_j$ such that $t \in \tau$.

If an execution trajectory τ is in M , it is said to satisfy the specification.

I. Monotone Specifications

We define the monotonicity property of a specification to describe the specification satisfaction by trajectories traversing through states that are greater in a defined partial ordering.

Definition 11 (Monotone Specification): A specification M for a transition system $S = (X, T, L, Y, H)$ is said to be a monotone specification iff the following condition is satisfied. Whenever a trajectory $\tau := x_0 \ell_0 x_1 \ell_1 \dots \ell_{N-1} x_N \in M$ then any trajectory $\tau' := x'_0 \ell'_0 x'_1 \ell'_1 \dots \ell'_{N-1} x'_N$, where $\tau' \succeq_\tau \tau$, i.e.,

$$x'_0 \succeq x_0, x'_1 \succeq x_1, \dots, x'_N \succeq x_N$$

also belongs to M .

J. Solver

We define a solver as an oracle that manifests a trajectory (also referred to as a plan) that could be executed on a given transition system to satisfy a provided specification. For example, a solver can be an optimization engine, a constraint solver, a heuristic-driven algorithm, etc. In general, in addition to the constraints imposed by the transition system and the provided specification, a solver may be assigned a limited amount of computational resources and time.

Definition 12 (Solver): Given a specification M for a transition system S , an initial state x_0 and a maximum compute time $t_{\text{compute}}^{\max}$, a solver Ω is an oracle that spends compute time $t_{\text{compute}}^\Omega \in (0, t_{\text{compute}}^{\max})$ to search for a specification satisfying execution trajectory $\tau \in M$ and returns it if found.

K. Objective Function

While the satisfaction of a specification ensures that the planning goal is achieved, a proposed plan's optimality can be evaluated using an objective function defined as follows.

Definition 13 (Objective Function): For a monotone transition system S with partial ordering $\succeq$, a function $f_S : \zeta_{\Sigma(S)} \rightarrow \mathbb{R}$ is said to be an objective function if for all trajectories $\tau_1, \tau_2 \in \Sigma(S)$ such that $\tau_1 \succeq_\tau \tau_2$, the objective $f_S(\zeta_{\tau_1}) \leq f_S(\zeta_{\tau_2})$.

IV. PLANNING PROBLEM IN TRANSITION SYSTEMS

A. Planning Problem

In this section, we use the defined mathematical concepts to formally state the planning problem in transition systems. Let $S = (X, T, L, Y, H)$ denote an infinite state transition system with a properly defined set of states X , transitions T , and an output map H . Let $\Xi = \{\hat{Y}_1, \hat{Y}_2, \dots, \hat{Y}_{N_y}\}$ denote a provided task site assignment that needs to be satisfied. The goal is to find an implementation $\zeta : (0, \gamma) \rightarrow X$ of some duration $\gamma \geq 0$, such that the image of its output behavior, $\xi_\zeta : (0, \gamma) \rightarrow Y$ contains the task sites in the assignment, i.e., $\hat{Y}_i \subset \text{Image}(\xi_\zeta)$ for $i = 1, 2, \dots, N_y$. Further, an objective function, $f_S : \zeta_{\Sigma(S)} \rightarrow \mathbb{R}$, must evaluate the plan optimality when implemented in the system S . Therefore, we state the planning problem as an optimization problem as follows.

$$\begin{aligned} & \text{minimize} && f_S(\zeta), \\ & \text{s.t. } \forall i && \hat{Y}_i \subset \text{Image}(\xi_\zeta) \end{aligned} \quad (2)$$

where Ξ_N is the task site assignment.

Solving the optimization problem in Eq. (2) is computationally expensive due to the infinite state space of the system S . Solving for the optimal plan may be infeasible under limited computation resources. Therefore, we define a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$, creating a discrete state space for a solver to operate. Further, for the task site assignment Ξ , we define a set of key state and key transition classes for the sampled transition system S_d . Then a specification $M \subset \Sigma(S)$ can be defined using the key state and key transition classes. Now, the problem reduces to finding an execution trajectory $\tau \in M$, such that its implementation ζ_τ minimizes the objective function. To this end, we redefine the optimization problem in Eq. (3) such that solving it yields a specification M satisfying plan in the sampled system.

$$\text{minimize}_{\tau \in M} f_S(\zeta_\tau). \quad (3)$$

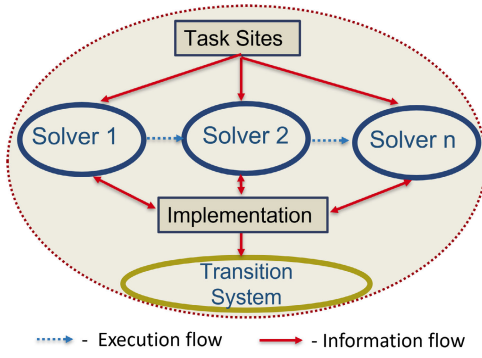


Fig. 5. Problem space for iterative planning: Solvers are called iteratively with the task site assignment and the current implementation to generate a better plan to be implemented on the transition system.

B. Solution Framework

Solvers with different capabilities produce solutions with varying levels of sub-optimality using varying computation times. Therefore, we propose a solution framework that enables an iterative solution optimization procedure by creating a problem space that allows multiple solvers to operate and solve the planning problem to formulate better solutions. The proposed framework is shown in Figure 5. In the problem space, the original infinite state system S and task site assignment Ξ that needs to be satisfied are defined. The goal is to find an implementation in the infinite state system to satisfy Ξ . The plan computation is done by invoking each solver iteratively for continual improvement. The proposed framework enables invoking faster solvers earlier to find a feasible solution and then continue to use the available computation resources to optimize the solution further.

Each solver in the problem space may operate in its own discrete or bounded and continuous state spaces of a sampled transition system S_d and specification M to find an execution trajectory. However, the proposed iterative planning scheme requires that each sampled transition system must agree on the feasibility of the plan provided by the previous solver and its satisfaction of the specification.

1) Specification Satisfaction: When a solver Ω is assigned $t_{\text{compute}}^{\Omega}$ computation time to solve the optimization problem, it will compute a plan τ as the solution. However, bounded computation time may result in a sub-optimal solution in terms of the objective function. Further, solving the optimization problem in Eq. (3) yields a plan in the corresponding sampled transition system. Therefore, we state the following theorem on specification satisfaction.

Theorem 1 (Specification Satisfaction): Given a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ of an infinite state transition system $S = (X, T, L, Y, H)$, and a specification $M \subset \Sigma(S_d)$ for a task site assignment Ξ , if a plan (trajectory) $\tau \in \Sigma(S_d)$ satisfies M , then its implementation ζ has the output behavior ξ_{ζ} that satisfies the task site assignment Ξ .

Proof: For the given task site assignment Ξ , let $\hat{X}_i$ for $i = 0, \dots, N_x$ denote the key states classes and $\hat{T}_j$ for $j = 0, \dots, N_t$ denote the key transition classes. If a plan τ satisfies $M \subset \Sigma(S_d)$ then $\tau \in M$. From the definition of the specification, any trajectory $\tau \in M$ must contain some state $x \in \hat{X}_i$ for all $i = 0, \dots, N_x$ and some transition $t \in \hat{T}_j$ for

all $j = 0, \dots, N_t$. From the definition of key states and key transitions, for an implementation ζ of any trajectory $\tau \in M$, there exists a $\gamma \geq 0$ such that $H(\zeta(\gamma)) \in \hat{Y}_i$ holds for all $\hat{Y}_i \in \Xi$. Therefore, the output behavior of ζ contains the task sites in its image, i.e., $\hat{Y}_i \subset \text{Image}(\xi_{\zeta})$ for $i = 0, \dots, N_x + N_y$. Thus, implementation of any trajectory $\tau \in M$ yields an output behavior ξ_{ζ} , satisfying the task site assignment Ξ . ■

Therefore, any solver that operates on a sampled transition system can find solutions that satisfy the task site assignment Ξ when implemented in the infinite state system.

2) Ability of Improvement: As the solvers operate on a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$, with a bounded computation time, a plan produced by the solvers could be sub-optimal in terms of the objective function f_S . Therefore, for a produced trajectory τ , if an improvement $\tau' \geq \tau$ exists such that $x'_i \in \tau_d$ for any state $x_i \in \tau$ where $x'_i \geq x_i$, the transitions in τ' must exist in T_d under the replacement of x_i by x'_i . Thus, for any $t_{ij} = (x_i, \ell, x_j) \in \tau$ and $x'_i \geq x_i$, a $t'_{ij} = (x'_i, \ell, x_j) \tau'$ must exist in T_d . To this end, we state the following theorem on the monotonicity of the sampled transition systems.

Theorem 2 (Monotonicity in Sampled Transition System): Given a monotone transition system $S = (X, T, L, Y, H)$ with respect to a partial ordering $\geq$, a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ of S is also monotone with respect to $\geq$.

Proof: For a transition system $S = (X, T, L, Y, H)$, the monotonicity condition states that the following is true for every $x_1, x'_1 \in X$ and $\ell \in L$, such that $(x_1, \ell, x'_1) \in T$: For all $x_2 \in X$ such that $x_2 \geq x_1$, there exists a transition $(x_2, \ell, x'_1) \in T$. From the definition of sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$, it is known that $X_d \subset X$, and $T_d \subset T$. For all $x_1, x'_1, x_2 \in X_d$ and $\ell \in L$, such that $(x_1, \ell, x'_1) \in T_d$ and $x_2 \geq x_1$, a transition $t^d = (x_2, \ell, x'_1)$ exists as $x_1, x'_1, x_2 \in X$ where $S = (X, T, L, Y, H)$ is monotone. As the transition, t^d exists in T_d , the sampled transition system S_d is monotone. ■

C. Iterative Planning

In the proposed iterative planning framework, the solvers work on finding a plan satisfying the specification. Each solver takes in the specification and creates a sampled transition system instance in the solver space. Then, it computes the solution as a trajectory and returns it to the infinite state system as an implementation. When the next solver is called for plan computation, it must identify the current implementation ζ that satisfies the task site assignment and corresponds to a specification satisfying trajectory in the solver's sampled transition system. To this end, we state the following theorem on the feasibility of the current implementation through iterative planning instances.

Theorem 3 (Recursive feasibility): Given a monotone transition system $S = (X, T, L, Y, H)$ and an implementation ζ , whose output behavior satisfies a task site assignment $\Xi = \{\hat{Y}_1, \hat{Y}_2, \dots, \hat{Y}_N\}$, let $S_d = (X_d, T_d, L, Y_d, H)$ be a sampled transition system of S , and M be a specification defined from Ξ . For any set of time stamps $\Gamma = \{\gamma_0, \dots, \gamma_{\mu}\}$, if $\{x \mid \exists \gamma \in \Gamma, x = \zeta(\gamma)\} \subseteq X_d$, then there exists a trajectory $\tau \in \Sigma(S_d)$ such that τ satisfies M .

Algorithm 1 Iterative Planning

Data: S - Infinite state transition system, Ξ - Task site assignment, x_0 - Initial state, $(\Omega_1, \dots, \Omega_n)$ - Sequence of solvers, $t_{compute}^{total}$ - Max total compute time, γ_H - Horizon length

Result: ζ_{final} - Final plan implementation

```

1  $i \leftarrow 1$ ;
2 Set  $S_{d_i}$  to a sampled transition system of  $S$ ;
3 Define specification  $M \in \Sigma(S_{d_i})$  from  $\Xi$ ;
4 Get current time  $t_{start} = clock()$ ;
5  $t_{compute}^i \leftarrow t_{compute}^{total} + t_{start} - clock()$ ;
6 if  $t_{compute}^i > 0$  then
7   Call  $\Omega_i$  with compute time  $t_{compute}^i$  to find a plan
    $\tau_{d_i} \in M$  starting from  $x_0$  for a horizon of  $\gamma_H$ ;
8   Set  $\zeta_i \leftarrow \zeta_{\tau_{d_i}}$ ;
9 for  $i = 2, \dots, n$  do
10  Define  $S_{d_i}$  such that  $\{x \mid \exists \gamma \in \Gamma \ x = \zeta(\gamma)\} \subseteq X_{d_i}$ 
   where the set of time stamps  $\gamma^d = \gamma_0^d, \dots, \gamma_N^d$ ;
11  Define the specification  $M \in \Sigma(S_{d_i})$  from  $\Xi$ ;
12   $t_{compute}^i \leftarrow t_{compute}^{total} + t_{start} - clock()$ ;
13  if  $t_{compute}^i > 0$  then
14    Call  $\Omega_i$  with compute time  $t_{compute}^i$  to find a plan
     $\tau_{d_i} \in M$  starting from  $x_0$  for a horizon of  $\gamma_H$ ;
15    if  $f_S(\zeta_{\tau_{d_i}}) \leq f_S(\zeta_{i-1})$  then
16      Set  $\zeta_i \leftarrow \zeta_{\tau_{d_i}}$ ;
17    else
18      Set  $\zeta_i \leftarrow \zeta_{i-1}$ ;
19  else
20    break;
21 return  $\zeta_{final} = \zeta_i$ ;

```

Proof: For a given transition system $S = (X, T, L, Y, H)$, and any implementation ζ , let $x_i = \zeta(\gamma_i) \in X$ for $\gamma_i \in \Gamma$. Let time durations $\gamma_i^d = (\gamma_{i+1} - \gamma_i) \in L$ for $i \in \{0, \dots, \mu - 1\}$. Then a corresponding trajectory to ζ in S can be defined as $\tau = x_0 \gamma_0^d x_1 \gamma_1^d \dots \gamma_{\mu-1}^d x_\mu \in \Sigma(S)$. However, for a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ sampled from S , the existence of $\tau \in \Sigma(S_d)$ requires that each $x_i \in \tau$ must be in X_d . Therefore, if $\{x \mid \exists \gamma \in \Gamma \ x = \zeta(\gamma)\} \subseteq X_d$ for some Γ , we see that there exists a $\tau \in \Sigma(S_d)$.

If M is a specification in S_d defined from Ξ , any $\tau \in M$ must contain states from each key state class and transition from each key transition class. From the definition of key state classes, let a subset of task site allocation, $\Xi_x = \{\hat{Y}_j \in \Xi \mid \hat{Y}_j \cap Y_d \neq \emptyset\}$ form key state classes in S_d . For a given implementation ζ that satisfies a task site assignment Ξ , there exists some $\gamma_j \geq 0$ such that $H(\zeta(\gamma_j)) \in \hat{Y}_j$ for all $j = 1, \dots, N$. Let Γ be defined such that $\{\gamma_j \mid \exists \hat{Y}_j \in \Xi_x H(\zeta(\gamma_j)) \in \hat{Y}_j\} \subseteq \Gamma$. For $\gamma_i \in \Gamma$ and time durations $\gamma_i^d = (\gamma_{i+1} - \gamma_i) \in L$ for $i \in \{0, \dots, \mu - 1\}$, a corresponding trajectory to ζ defined as $\tau = x_0 \gamma_0^d x_1 \gamma_1^d \dots \gamma_{\mu-1}^d x_\mu$ contains states from each key state class, i.e. there exists $\gamma_j \in \Gamma$, such that $\zeta(\gamma_j) \in \tau$ and $H(\zeta(\gamma_j)) \in \hat{Y}_j$ for all $\hat{Y}_j \in \Xi_x$.

Let $\Xi_t = \Xi \setminus \Xi_x$ be the set of task sites that do not form key state classes in S_d , i.e., $\Xi_t = \{\hat{Y}_j \in \Xi \mid \hat{Y}_j \cap Y_d = \emptyset\}$. Therefore, they form key transition classes. From the given existence of implementation ζ that satisfies Ξ , there exists some $\gamma_j \geq 0$ such that $H(\zeta(\gamma_j)) \in \hat{Y}_j$ for all $\hat{Y}_j \in \Xi_t$. Therefore, a corresponding trajectory to ζ defined as $\tau = x_0 \gamma_0^d x_1 \gamma_1^d \dots \gamma_{\mu-1}^d x_\mu$ contains transitions from each key transition class by definition. Since $\tau \in \Sigma(S_d)$ contains states from each key state class and transitions from each key transition class, it satisfies M . Thus, a trajectory $\tau \in \Sigma(S_d)$ exists such that $\tau = x_0 \gamma_0^d x_1 \gamma_1^d \dots \gamma_{N-1}^d x_N$ satisfies M . ■

Theorem 3 establishes that given the set of states, X_d of a sampled transition system includes states sampled from an implementation that satisfies a task site assignment, there exists a specification-satisfying trajectory in the sampled transition system of the solver. This ensures the existing plan remains feasible for each sampled transition system when solvers are iteratively called through the planning framework.

From Theorem 3, each solver in the iterative planning framework can compute solutions while the current implementation remains feasible. Each solver employed in the framework can be invoked to generate solutions with better objective values. To this end, we propose Algorithm 1.

Iterative planning takes the transition system S , task site assignment Ξ , initial state x_0 and a set of solvers $\{\Omega_1, \Omega_2, \dots, \Omega_n\}$. Each solver is allowed the maximum computation time. At each step of planning in the iterative framework, a sampled transition system S_d and a specification are created such that Theorem 3 is satisfied.

Corollary 1: Let S be a transition system and Ξ be a task site assignment. Let Ω_i s.t. $i > 1$ be a solver in Algorithm 1 and S_d^i be the corresponding sampled transition system. For an implementation ζ_{i-1} found from the previous solver Ω_{i-1} , there exists an implementation $\zeta_i \in \zeta_{\Sigma(S_d^i)}$ whose output behavior satisfies Ξ .

Proof: This is a direct consequence of Theorem 3. In Algorithm 1, each S_d^i is sampled such that 3 is satisfied. It guarantees the existence of a $\tau \in \Sigma(S_d^i)$ such that the output behavior of its implementation $\zeta_i \in \zeta_{\Sigma(S_d^i)}$ satisfies Ξ . ■

This ensures the recursive feasibility of the planning framework. Then, the corresponding solver is allowed the given computation time for plan computation. If the computed plan is better than the current plan, the current plan is updated to the computed plan. Once all the solvers are iterated or the allowed compute time runs out, the current plan is returned for implementation. Note that in Algorithm 1, $clock()$ indicates the Timer function.

The planning scheme described in Algorithm 1 has several advantages compared to the direct execution of a plan from a single solver.

- 1) Multiple solver integration - Provided that the corresponding sampled transition system satisfies Theorem 3, any solver can be interfaced seamlessly with the planning framework. This enables a versatile implementation that can iterate over solvers with varying sampled state spaces.
- 2) Compute time constraints - Allocation of compute time for each solver as well as for the total planning scheme

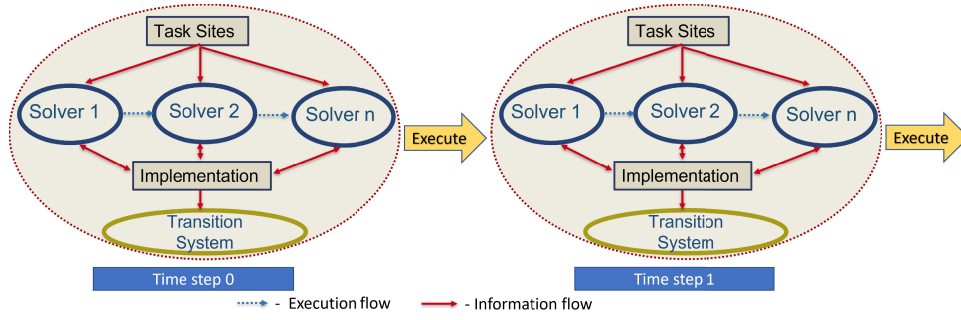


Fig. 6. Shrinking horizon execution: At each step of execution, a problem space is created to run iterative planning.

allows time-sensitive planning tasks to be completed with the planning framework.

- 3) Continual improvement - At each iterative call to the solvers, an improvement constraint is imposed on the next solution. This enables continuous improvement of the solution toward optimality.
- 4) Implementable solution - At each step of planning, an implementation in the original system that satisfies the task site assignment is always at hand. If the next solver called iteratively cannot provide a better plan, the existing plan can be directly executed on the infinite state system to satisfy the task site allocation.

Next, we describe the shrinking horizon execution followed for the plan produced by Algorithm 1.

D. Shrinking Horizon Execution

After each iterative planning step, the produced plan is executed step by step in a shrinking horizon fashion following the execution scheme shown in Figure 6. Algorithm 2 describes the proposed execution scheme. At each planning step, Algorithm 1 is called to compute a plan for the current specification. Once a plan with a better objective is computed, the plan is executed a time step forward. If a task site $\hat{Y}_j$ was visited at the execution of the current step, the assignment is updated for the next step by excluding $\hat{Y}_j$ from the task site assignment. As Theorem 3 ensures the recursive feasibility of the current plan, it can always be executed in the system if a better plan does not exist. In the next section, we describe an example implementation of the proposed solution framework.

V. APPLICATION: ENERGY-AWARE ROUTE PLANNING

In this section, we introduce an example planning problem and solve it using the techniques of this paper. Consider a scenario where a collaborative team of vehicles is assigned a set of tasks to be completed at different locations on a known map. The objective is to find the optimum route plan for each vehicle such that the overall plan minimizes the total time to complete all assigned tasks. This problem can be viewed as a VRP, an extension of the TSP. However, complexities arise when the vehicles are of different types, like UAVs or UGVs with different capabilities. Further, when the vehicles have limited energy, the route planner must account for energy consumption and properly handle recharging. Therefore, this

Algorithm 2 Shrinking Horizon Execution

Data: S -Infinite state transition system, Ξ_0 - Task site assignment, x_0 - Initial state, $k \in (1, \dots, K)$ - Execution time steps, $t_{compute}^{total} (< \gamma_{step})$ - Max total compute time for each step, γ_{step} - Duration of a time step.

Result: ζ_{final} - Executed implementation

```

1 for  $k = 1, \dots, K$  do
2   Create Iterative Planning instance  $k$  using system  $S$ 
   and task site assignment  $\Xi_{k-1}$ ;
3   Set initial state  $x_{k-1}$ ;
4   Run Iterative Planning for  $\Xi_{k-1}$  on system  $S$  starting
   from  $x_{k-1}$  for a horizon of  $\gamma_H = (K - k + 1) \times \gamma_{step}$ 
   with compute time  $t_{compute}^{total}$ ;
5   Get  $\zeta^*$  from Iterative Planning output;
6   if  $k = 1 \vee f_S(\zeta^*) \leq f_S(\zeta_{k-1})$  then
7      $\zeta_k \leftarrow \zeta^*$ ;
8   else
9      $\zeta_k \leftarrow \zeta_{k-1}$ ;
10  Set  $x_k = \zeta_k(\gamma_{step})$  after execution for  $\gamma_{step}$  duration ;
11  Let  $\hat{Y}_j$  for  $j \in \{1, \dots, j_{k-1}\}$  be task sites of  $\Xi_{k-1}$ ;
12  Get satisfied task sites,
    $\Phi = \{\hat{Y}_j \in \Xi_{k-1} \mid \exists \gamma \leq \gamma_{step} \text{ s.t. } H(\zeta_k(\gamma)) \in \hat{Y}_j\}$ ;
13  Shift  $\zeta_k$  by  $\gamma_{step}$ , i.e.,  $\zeta_k(\gamma) \leftarrow \zeta_k(\gamma - \gamma_{step})$ ;
14  Exclude satisfied task sites and update assignment,
    $\Xi_k = \Xi_{k-1} \setminus \Phi$ ;
15 return  $\zeta_{final} = \zeta_K$ ;

```

example represents a generalized energy-aware cooperative task site assignment.

A. Running Example

Consider a known 2D map of an area of interest, $G = [0, 20] \times [0, 20]$. Let $G_R \subset G$ denote a road and three depot locations at the ends as in Figure 7. The task is to monitor the road using a team of UAVs and a UGV, where each vehicle has the required perception abilities. Also, the vehicles have limited energy and can be recharged if required. The depots, A , B and C , are equipped with charging pads for wireless charging of UAVs and a battery-swapping facility for the UGV. Further, the UGV is also equipped with a charging pad to conduct

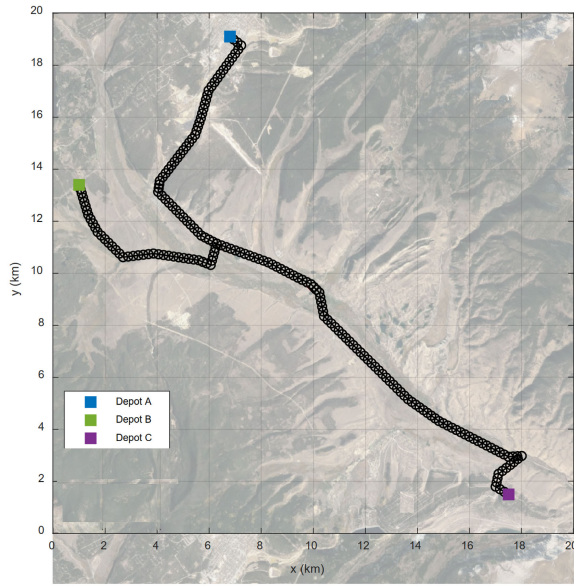


Fig. 7. Example map.

wireless charging of two UAVs at a time. We describe each vehicle in detail as follows.

1) *UGV Specifications*: The motion of the UGV is restricted to the road G_R denoted in black in Figure 7. It has a total usable energy of $B^{UGV} = 25.01$ MJ. The maximum velocity, $v_{g_{max}} = 4.5 \text{ ms}^{-1}$. The power consumption curve of the UGV in Watts is as follows,

$$Pe_g(v_g) = \begin{cases} 200\text{W}, & v_g = 0, \\ 1.05(464.8V_g + 356.3)\text{W}, & v_g > 0. \end{cases} \quad (4)$$

2) *UAV Specifications*: The UAVs can travel freely within the map G . They have limited energy storage onboard with a total of $B^{UAV} = 287.7$ kJ usable energy. The maximum UAV velocity is $v_{a_{max}} = 16 \text{ ms}^{-1}$. The power consumption curve of a UAV in Watts is as follows,

$$Pe_a(v_a) = 1.05(0.0461v_a^3 - 0.5834v_a^2 - 1.8761v_a + 229.6)\text{W}. \quad (5)$$

UAVs can dock for recharging on a UGV or a charging pad at a depot. The wireless power transfer rate in Watts when recharging follows the following curve, which depends on the UAV battery's current energy level (E_a in kJ).

$$Pe_{chg}(E_a) = \begin{cases} 310.8, & E_a \leq 270.4, \\ 17.965(287.7 - E_a), & 270.4 \leq E_a \leq 287.7 \end{cases} \quad (6)$$

We use the example map and the vehicle specifications as the selected application in this work. Each UAV can travel to any point in the 2D map G , and each UGV travels only along the road G_R . Further, the energy is consumed or replenished following the consumption and charging curves. The continuous states of spatial, energy, and time increase the complexity of the planning problem. When the number of vehicles increases, the number of state and search space dimensions increases. Planning problems like the running

TABLE I
VALUES OF THE STATUS FLAG f_j

f_j	Vehicle type	Status
0	UGV	No UAVs docked
1	UGV	1 UAV docked
2	UGV	2 UAVs docked
3	UAV	Not docked
4	UAV	Docked

example involve searching for solutions in multi-dimensional infinite-state systems. In this work, we apply the defined concepts of transition systems to mathematically represent the infinite state systems for the motion planning task. In the next section, we formulate the routing problem mathematically using defined preliminaries. Further, we analyze the results of the proposed solution when the problem is scaled in terms of the number of agents and the number of task sites.

B. Transition System Formulation

In this section, we mathematically formulate the transition system for the route planning problem in the running example scenario using the established theoretical framework. Let the team consist of J vehicles. We denote the 2D spatial coordinate of the j^{th} vehicle as $(p_x^j, p_y^j) \in G$, and the energy level as $e_j \in [0, B_{max}^j]$ where B_{max}^j is the battery capacity of vehicle j . Further, we define status flag f_j encoding the vehicle type (UGV/UAV) and its status. The value of f_j is assigned as shown in Table I.

Then, we define the state of the multi-vehicle system at time γ as follows.

$$x = [x_1, x_2, \dots, x_J, \gamma] \in X, \quad (7)$$

where $x_j = [p_x^j, p_y^j, e_j, f_j]$ for $j = 1, 2, \dots, J$ and X is the state space. We define a transition system for using the state and transitions representing state evolution as follows.

Definition 14 (Multi-vehicle System): Given a multi-vehicle system of J vehicles whose state is denoted as $x_j = [p_x^j, p_y^j, e_j, f_j]$ for $j = 1, 2, \dots, J$, a transition system $S = (X, T, L, Y, H)$ is defined as follows. (1) The state space, $X = \{x \mid x = [x_1, x_2, \dots, x_J, \gamma]\}$ where, $x_j = [p_x^j, p_y^j, e_j, f_j]$ for $j = 1, 2, \dots, J$ and γ is the time. (2) The set of transitions, $T = \{t \mid \exists \delta_\gamma > 0, \text{ s.t. } t = (x, \delta_\gamma, x') \text{ for } x, x' \in X\}$. (3) The set of labels, $L = \{\delta_\gamma > 0 \mid \exists t \in T, \text{ s.t. } t = (x, \delta_\gamma, x') \text{ for } x, x' \in X\}$. (4) Output space, $Y = \{y \mid y = [y_1, y_2, \dots, y_J]\}$ where, $y_j = [p_x^j, p_y^j]$ for $j = 1, 2, \dots, J$. (5) Output map, $H: X \rightarrow Y$.

Here, any transition $t = (x, \delta_\gamma, x') \in T$ is defined as an evolution of state from any time γ to $\gamma + \delta_\gamma$, where vehicles can change spatial coordinates, energy levels, and status, flags f_j , under their dynamics and power consumption curves. Each transition is assigned a label $\delta_\gamma \in L$. We set the output $y = H(x) = [y_1, y_2, \dots, y_J]$, where each $y_j = [p_x^j, p_y^j]$, which contains the spatial coordinates of the j^{th} agent of the system. Now, the transition system for the infinite state system

is fully defined as $S = (X, T, L, Y, H)$. Next, we discuss some required properties of the defined system.

C. Ordering Relation and Monotonicity

For a system $S = (X, T, L, Y, H)$, we introduce the partial ordering relation $\succeq$ as a measure of a state's goodness for comparison as follows.

Definition 15 (Partial Ordering): For a transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following 14, let any state be defined as, $x = [x_1, x_2, \dots, x_J, \gamma] \in X$ where, $x_j = [p_x^j, p_y^j, e_j, f_j]$ for $j = 1, 2, \dots, J$ and γ is the time. A binary relation $\succeq$ that is reflexive, anti-symmetric, and transitive is called partial ordering relation defined on X , if it satisfies the following. (1) For any $x, x' \in X$ such that $x' \succeq x$, $H(x) = H(x')$ must hold. This ensures that the spatial coordinates of all agents in x and x' are equal when comparing the energy and time dimensions.

(2) For any states $x, x' \in X$, let e_j, e'_j denote energy levels of any vehicle $j \in \{1, \dots, J\}$, respectively. If $x' \succeq x$ then $e'_j \geq e_j$ for all $j \in \{1, \dots, J\}$ (3) For any $x, x' \in X$, let γ, γ' denote time, respectively. If $x' \succeq x$, then $\gamma' \leq \gamma$.

Given the partial ordering, the system states can now be ranked by measuring their goodness using the energy level of the vehicles and the time. Further, with the partial ordering defined, the monotonicity of S follows from the Definition 4.

Corollary 2: A transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following Definition 14 is monotone with respect to a partial ordering $\succeq$ as defined in Definition 15.

Proof: Let any $x_1, x'_1 \in X$ and $\delta_\gamma \in L$, such that a transition $t = (x_1, \delta_\gamma, x'_1) \in T$ exists. Let $x_2 \in X$ such that $x_2 \succeq x_1$, that is x_2 ranks better than x_1 due to having higher energy level or earlier time. Because it is given that a transition $t = (x_1, \delta_\gamma, x'_1)$ exists, x'_1 is reachable from x_1 in time δ_γ . Therefore, x'_1 is reachable from a state with higher energy or earlier time than x_1 , like $x_2 \in X$. Thus, for any $x_2 \in X$ such that $x_2 \succeq x_1$, a transition $(x_2, \ell, x'_1) \in T$ always exists. Therefore, the transition system S defined from a multi-vehicle system following Definition 14 is monotone. ■

D. Task Site Assignment for Monitoring Tasks

In the running example, the goal is to monitor the road map G_R . Each road location forms classes in the output space Y to create the task site assignment.

Definition 16 (Task Site Assignment for Monitoring Tasks): For a transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following Definition 14, and a road G_R of N_R locations, each location $\hat{y}_i = (p_x^i, p_y^i) \in G_R$ forms an output class $\hat{Y}_i$ in the task site assignment such that $\hat{Y}_i = \{y \mid \exists y \in Y \text{ s.t. } \hat{y}_i \in y \text{ where } \hat{y}_i \in G_R\}$. The collection of all output classes is denoted as the task site assignment. $\Xi = \{\hat{Y}_1, \dots, \hat{Y}_{N_R}\}$.

Thus, output behavior ξ_ζ of any implementation ζ , such that $\text{Image } \xi_\zeta \cap \hat{Y}_i \neq \emptyset$ for all $i = 1, \dots, N_R$, satisfies the goal of road monitoring.

E. Planning Problem for Monitoring Tasks

Each implementation that satisfies the task site assignment is evaluated using the objective function. The goal of the planning task is to find a trajectory that satisfies the task site assignment when implemented while minimizing the total time spent executing the trajectory. Therefore, we define the objective function for monitoring tasks as follows.

Definition 17 (Objective Function for Multi-vehicle System): For a transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following Definition 14, let $\tau = x_0 \delta_\gamma^0 x_1 \delta_\gamma^1 \dots \delta_\gamma^{N-1} x_N \in \Sigma(S)$ denote a trajectory and ζ_τ be its implementation. An objective function $f_S: \zeta_\Sigma \rightarrow \mathbb{R}$, is defined to return the time of the final state, i.e., $\gamma_N = f_S(\zeta_\tau)$ where $\gamma_N \in x_N$.

The goal is to find an implementation $\zeta: (0, \gamma) \rightarrow X$ of some duration $\gamma \geq 0$, such that the image of its output behavior, $\xi_\zeta: (0, \gamma) \rightarrow Y$ contains the task sites in the assignment, i.e., $\hat{Y}_i \subset \text{Image}(\xi_\zeta)$ for $i = 1, 2, \dots, N_R$. The objective function, $f_S: \zeta_\Sigma(S) \rightarrow \mathbb{R}$, evaluates the plan optimality when implemented in the system S . Therefore, we state the planning problem as an optimization problem as follows.

$$\begin{aligned} & \text{minimize} && f_S(\zeta), \\ & \text{s.t. } \forall i && \hat{Y}_i \subset \text{Image}(\xi_\zeta) \end{aligned} \quad (8)$$

To solve this problem optimally, the best trajectory in a continuous multi-dimensional state space must be found. However, real-time planning scenarios make this search infeasible. Therefore, we define sampled transition systems for each solver space to efficiently use Algorithm 1 for plan generation and Algorithm 2 for execution.

F. Sampled Transition Systems and Specifications for Monitoring Tasks

Each transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following Definition 14, has a continuous state space. We define sampled transition systems $S_d = (X_d, T_d, L, Y_d, H)$ following the Definition 7. From the Monotonicity Theorem 2, we have the following result on sampled transition systems.

Corollary 3: For a transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following Definition 14, a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ of S is monotone.

Proof: From Corollary 2, we have that a transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following Definition 14 is monotone. Further, from Theorem 2, we have that any transition system sampled from a monotone transition system is monotone. Therefore, a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ is monotone. ■

Given a task site assignment Ξ and a sampled transition system $S_d = (X_d, T_d, L, Y_d, H)$ of S , a specification $M \subset \Sigma(S_d)$ can be defined following Definition 10. For the defined specification M , we state the following result on the monotonicity of M .

Corollary 4: Given a transition system $S = (X, T, L, Y, H)$ defined from a multi-vehicle system of J vehicles following Definition 14 and a task site assignment Ξ , the

specification $M \subset \Sigma(S)$ defined from the task site assignment Ξ , is monotone.

Proof: Let $\tau = x_0 \delta_\gamma^0 x_1 \delta_\gamma^1 \cdots \delta_\gamma^{N-1} x_N \in \Sigma(S)$ denote a trajectory in M whose implementation is ζ . Let ξ_ζ be the output behavior of ζ . Since, $\tau \in M$, ξ_ζ satisfies Ξ . Let any trajectory $\tau' = x'_0 \ell_0 x'_1 \ell_1 \cdots \ell_{N-1} x'_N$, where

$$x'_0 \geq x_0, x'_1 \geq x_1 \cdots, x'_N \geq x_N.$$

That is, each x'_i , which has higher energy and earlier time, ranks better than x_i . From the definition of $\geq$ we have $H(x'_i) = H(x_i)$ for each i . Therefore, output behavior ξ' of an implementation of τ' satisfies the task site assignment Ξ . Thus, τ' is in M . Hence, the specification M is monotone. ■

From Corollary 3 and Corollary 4, we state that improvements for states are possible in the sampled transition systems defined from multi-vehicle systems, and when such improvements yield improved trajectories, they will still satisfy the intended specification. Therefore, we have established the possibility of improvement for a specification-satisfying plan.

Using the sampled transition systems S_d and the defined specification $M \in \Sigma(S_d)$, we can restate the optimization problem in Equation 8 as follows.

$$\underset{\tau \in M}{\text{minimize}} f_S(\zeta_\tau), \quad (9)$$

From Theorem 1, we have that any $\tau \in M$, which is a solution to the optimization problem in Equation 9, yields an output behavior that satisfies the task site assignment from which M was defined. Further, if the sampling for the creation of S_d was conducted such that recursive feasibility conditions of Theorem 3 are satisfied, it ensures a specification-satisfying trajectory always exists in the sampled system.

Now, we have completely defined a planning problem in a multi-vehicle system using the mathematical concepts of transition systems. We have also proved the necessary conditions for using Algorithm 1 to iteratively employ a set of solvers to find a plan. Next, we present formulation of the solvers used in Algorithm 1 for the monitoring task.

G. Solver Formulation

In this section, we introduce the solvers used in the iterative planning framework and the respective sampled transition systems for these optimization solvers as follows for a given transition system for a multi-vehicle system S and a monitoring tasks site assignment Ξ .

1) Team-level discrete optimization

This solver operates in a sampled transition system S_{d_1} of the original system S and provides a feasible plan $\tau_{d_1} \in M_1$ for the whole team that satisfies the specification M_1 defined from Ξ .

2) Agent-level optimization

This solver operates in another sampled transition system S_{d_2} and attempts to find a solution $\tau_{d_2} \in M_2$ such that it is better than the previous solution, i.e., $f_S(\zeta_{d_1}) \geq f_S(\zeta_{d_2})$ where ζ_{d_1}, ζ_{d_2} are the respective implementations of τ_{d_1} and τ_{d_2} . Here, M_2 is the specification in S_{d_2} defined from Ξ . This instance of the solver operates at each agent level and optimizes the motion trajectory assigned

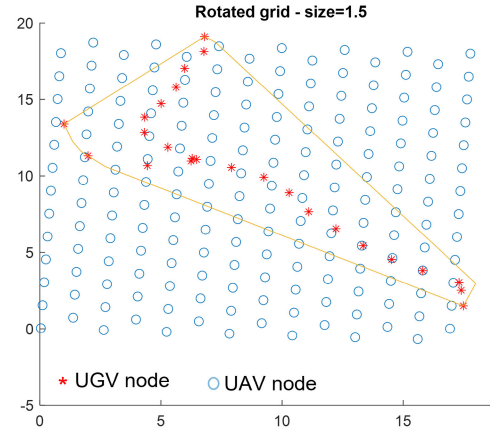


Fig. 8. Discretized example map. The region of operation is the spatial coordinates inside the convex hull.

by the team-level plan. Therefore, it operates in the same discrete spatial points. However, the energy and time dimensions are continuous.

Next, we look at the formulation of the previously described optimization solvers and their operating spaces.

H. Team-Level Discrete Optimization (TDO)

The goal of the TDO is to generate a feasible plan that satisfies the specification for the whole team using minimal computation resources and time. Therefore, we employ a sampling procedure to discretize the state space and time to formulate sampled transition system.

1) *Time Discretization:* The time discretization step γ_d is chosen to set the resolution of the planning horizon. This can be chosen arbitrarily to match the specifications of the problem. In the running example, we set $\gamma_d = 5 \text{ min}$.

2) *Spatial Discretization:* The spatial dimension consists of all possible points the vehicles can travel in the known map G . However, the UGVs are restricted to the road denoted as G_R . As the time step $\gamma_d = 5 \text{ min}$, we fix the velocity of the vehicles and evaluate the distance traveled by each vehicle in duration γ_d . For the UGVs, we discretize the road to a set of points $P_g = \{p_g | (p_g^x, p_g^y) \in G_R\}$, which are 1.2 km apart. The UAVs are capable of traveling between any points within the map G . Thus, we overlay a triangular grid of size 1.5 km on the map to sample a set of points in the 2D space. We orient the grid such that maximum overlap is achieved with P_g to remove redundant points. Then, we crop the grid to include the points within the convex hull of the region of operation to further reduce the state space size to set $P_a = \{p_a | (p_a^x, p_a^y) \in G\} \cup P_g$. Here, $P_g \subset P_a$ allows UAVs to rendezvous with UGVs. The final spatial discretization is shown in Figure. 8.

3) *Energy Dimension Discretization:* The transition of vehicles within the spatial nodes depletes their energy following the consumption curves in Eq. (4) and (5). The recharging of the UAV batteries either on a depot or on a ground vehicle replenishes the UAV batteries following the curve in Eq. (6). Further, a UGV battery depletes following the same curve

when recharging UAVs. However, from the time discretization, it is known that each depletion and recharge task is continued for a γ_d interval per step. We calculate energy depletion per moving step for each vehicle assuming constant velocity for a γ_d duration as $Pe_a(v_a) \times \gamma_d$ for UAVs and $Pe_g(v_g) \times \gamma_d$ for UGVs. Let the number of maximum energy levels of UAV be $B_a^{\max}$ and that of UGV be $B_g^{\max}$. We find the number of energy levels depleted as,

$$B_{\text{move}}^a = \lceil \frac{B^{\text{UAV}}}{B_a^{\max}} \times Pe_a(v_a) \gamma_d \rceil$$

for the UAV, and

$$B_{\text{move}}^g = \lceil \frac{B^{\text{UGV}}}{B_g^{\max}} \times Pe_g(v_g) \gamma_d \rceil$$

for the UGV. Then, we find the energy levels gained per charging step of γ_d duration for the UAVs as

$$B_{\text{charge}}^a = \lfloor \frac{B^{\text{UAV}}}{B_a^{\max}} \times Pe_{\text{chg}} \gamma_d \rfloor.$$

At each charging step on the ground vehicle, B_{charge}^a is depleted from the UGV energy. We always over-approximate energy depletion through *ceil* operation and under-approximate the energy gain by *floor* operation.

Given the above discrete formulation of the state space, we define the optimization problem as an SMT problem to search for a feasible solution. This enables computing a feasible solution faster, as searching for the optimal solution is NP-hard. To this end, we define the set of constraints for the SMT problem that encodes the vehicle motion dynamics, energy dynamics, and specification satisfaction. We look at the formulation of such a constraint set for a team of one UGV and two UAVs.

Let a K be the maximum length of the allowed time horizon. For each $k = 1, \dots, K$ we define the following variables to formulate the SMT problem.

1. $p_a^i(k) \in P_a$ - Position of UAV i at time step k .
2. $p_g(k) \in P_g$ - Position of UGV at time step k .
3. $b_a^i(k) \in \{0, \dots, B_a^{\max}\}$ - Battery level of UAV i at time step k .
4. $b_g(k) \in \{0, \dots, B_g^{\max}\}$ - Battery level of UGV at time step k .
5. $v_a^i(k) \in \{True, False\}$ - Boolean variable denoting status of motion of UAV i at time step k .
6. $v_g(k) \in \{True, False\}$ - Boolean variable denoting status of motion of UGV at time step k .
7. $s^i(k) \in \{True, False\}$ - Boolean variable denoting charging status of UAV i on the UGV at time step k .

Using the defined variables, we state the following constraints on the motion of the vehicles. For the ground vehicle,

$$\begin{aligned} \forall p, \forall k \quad (p_g(k) = p) \wedge v_g(k) \\ \implies \bigvee_{q \in P_g^{\text{next}}(p)} p_g(k+1) = q. \end{aligned} \quad (10)$$

Here, $P_g^{\text{next}}(p) \subset P_g$ denotes the next possible positions for a UGV currently at p . Similarly, we state the dynamical

constraint for the i^{th} UAV as,

$$\begin{aligned} \forall p, \forall k \quad (p_a^i(k) = p) \wedge v_a^i(k) \wedge \neg(s^i(k) \wedge s^i(k+1)) \\ \implies \bigvee_{q \in P_a^{\text{next}}(p^i)} p_a^i(k+1) = q. \end{aligned} \quad (11)$$

Here, $P_a^{\text{next}}(p) \subset P_a$ denotes the next possible positions for a UAV i currently at p . Further, the UAV motion is described by the following constraints accounting for charging on UGV.

$$\begin{aligned} \neg v_a^i(k) \wedge \neg(s^i(k) \wedge s^i(k+1)) \\ \implies p_a^i(k+1) = p_a^i(k), \\ s^i(k) \implies p_a^i(k) = p_g(k). \end{aligned} \quad (12)$$

Then, we state the constraints on the energy dynamics. UGV energy at time step k , depends on its velocity $v_g(k)$ and the status of UAV charging on UGV at k . We state the UGV energy dynamics as follows for the two UAV systems in focus.

$$\begin{aligned} \neg v_g(k) \wedge (s^1(k) \wedge s^2(k)) \\ \implies b_g(k+1) = b_g(k) - 2B_{\text{charge}}^a, \end{aligned} \quad (13)$$

$$\begin{aligned} \neg v_g(k) \wedge (s^1(k) \vee s^2(k)) \\ \implies b_g(k+1) = b_g(k) - B_{\text{charge}}^a, \end{aligned} \quad (14)$$

$$\begin{aligned} \neg v_g(k) \wedge (\neg s^1(k) \wedge \neg s^2(k)) \\ \implies b_g(k+1) = b_g(k), \end{aligned} \quad (15)$$

$$\begin{aligned} v_g(k) \wedge (s^1(k) \wedge s^2(k)) \\ \implies b_g(k+1) = b_g(k) - 2B_{\text{charge}}^a - B_{\text{move}}^g, \end{aligned} \quad (16)$$

$$\begin{aligned} v_g(k) \wedge (s^1(k) \vee s^2(k)) \\ \implies b_g(k+1) = b_g(k) - B_{\text{charge}}^a - B_{\text{move}}^g, \end{aligned} \quad (17)$$

$$\begin{aligned} v_g(k) \wedge (\neg s^1(k) \wedge \neg s^2(k)) \\ \implies b_g(k+1) = b_g(k) - B_{\text{move}}^g. \end{aligned} \quad (18)$$

Similarly, we state the constraints for energy dynamics for the UAV i as follows. Let Dp^i be a Boolean variable denoting whether the UAV i is at a depot location. Here, $p_{\text{depot}_1}, p_{\text{depot}_2}, p_{\text{depot}_3}$ denotes the discrete nodes corresponding to depot locations.

$$\begin{aligned} Dp^i = (p_a^i(k) = p_{\text{depot}_1} \vee p_a^i(k) = p_{\text{depot}_2} \\ \vee p_a^i(k) = p_{\text{depot}_3}). \end{aligned} \quad (19)$$

The following constraints state the energy dynamics of the UAV when moving, charging on a UGV, or charging at a depot location.

$$\neg s^i(k) \wedge \neg v_a^i(k) \wedge \neg Dp^i \implies b_a^i(k+1) = b_a^i(k), \quad (20)$$

$$\begin{aligned} \neg s^i(k) \wedge v_a^i(k) \wedge \neg Dp^i \\ \implies b_a^i(k+1) = b_a^i(k) - B_{\text{move}}^a, \end{aligned} \quad (21)$$

$$\begin{aligned} (s^i(k) \vee Dp^i) \wedge \neg v_a^i(k) \\ \implies b_a^i(k+1) = b_a^i(k) + B_{\text{charge}}^a, \end{aligned} \quad (22)$$

$$\begin{aligned} (s^i(k) \vee Dp^i) \wedge v_a^i(k) \\ \implies b_a^i(k+1) = b_a^i(k) + B_{\text{charge}}^a - B_{\text{move}}^a. \end{aligned} \quad (23)$$

From the discretization of the spatial dimension, the original task of visiting G_R reduces to a specification of visiting all discrete positions in P_g . Thus, we state our monitoring task as a constraint as follows.

$$\forall p \in P_g \quad \bigvee_{k=0}^K (p_g(k) = p \vee \bigvee_{\forall i} p_a^i(k) = p) = \text{True}. \quad (24)$$

This constraint is satisfied for each location in P_g , when it is visited by any vehicle at any time step before K . Therefore, its satisfaction ensures the satisfaction of the task site assignment. Now, any solution τ that satisfies the previously defined set of constraints is a feasible solution to the Team-level discrete optimization problem. We employ Z3, an SMT solver, to search for a feasible solution faster.

The feasible solution that we gain after TDO is sub-optimal due to three kinds of sub-optimality introduced as follows.

1. Sub-optimality of the SMT solver.
The SMT solver only searches for constraint satisfaction. Therefore, the produced solution is not optimal.
2. Sub-optimality due to discretization.
The state space is a discrete space generated through the sampling process. Each state transition takes place in an interval γ_d . Therefore, the solution trajectories could be sub-optimal when compared to continuous time, and continuous state trajectories.
3. Sub-optimality due to conservativeness of the sampled system.
The sampled system is generated by assuming a constant velocity of the vehicles. However, when a generated plan is executed in the original infinite state system, a suitable optimal velocity can be selected by the lower-level control algorithm. Further, due to the over-approximations of the energy depletion and under-approximation of energy gained while charging, the sampled system is more conservative than the original infinite state system. This introduces further sub-optimality to the TDO solution.

TDO is used to generate a feasible solution quickly. Thus, the remainder of allocated computation resources and time can be exploited toward reducing the sub-optimality of the solution. To this end, we use agent-level optimization as the next solver in the iterative planning framework, where we further optimize the solution generated by TDO.

I. Agent-Level Optimization (AO)

We identify that the TDO solution τ_{d_1} can be optimized at each agent level in between events involving multiple agents. From the TDO solution, we make the following observations.

1. The traversal order of the visits to task site coordinates can be modified.
2. All charging events can be dropped or reordered, as long as the energy dynamic constraints are not violated.
3. Charging on the UGV is equivalent to charging at depots. However, they are only active at a time step where the UGV is at that specific spatial coordinate.

Based on the previous observations, improvements are possible to the AO solution τ_{d_2} , such that $f_s(\zeta_{d_1}) \geq f_s(\zeta_{d_2})$, where ζ_{d_1}, ζ_{d_2} are respective implementations of τ_{d_1} and τ_{d_2} .

For each UAV, we formulate a vehicle routing optimization problem as an SMT problem. Therefore, we operate within a set of discrete coordinates in the spatial dimension while allowing the time and energy dimensions to be continuous. To guarantee the recursive feasibility of ζ_{d_1} at AO, the conditions of Theorem 3 must be satisfied. For some set of time stamps $\Gamma = \{\gamma_0, \dots, \gamma_\mu\}$, let $\tilde{X} = \{x \mid x = \zeta_{d_1}(\gamma) \exists \gamma \in \Gamma\}$. Since time and energy dimensions are continuous, we only need to ensure that UAV can reach $\tilde{X}$ projected to the spatial dimension. Let $\tilde{Y} = \{y \mid \exists x \in \tilde{X} \ y \in H(x)\}$. We define the following variables for each UAV.

1. $N^v = \{N_1^v, \dots, N_n^v\}$ - The set of coordinates to be visited to satisfy the goal.
2. $N^d = \{N_1^d, \dots, N_r^d\}$ - The set of coordinates of depot locations where charging is possible.
3. $N^g = \{N_1^g, \dots, N_q^g\}$ - The set of coordinates of locations where charging is possible on the UGV.
4. $\gamma_1^g, \dots, \gamma_q^g$ - The set of timestamps where each UGV charging location is active.
5. $\rho(k) \in N^v \cup N^d \cup N^g \cup \tilde{Y}, k = 0, \dots, K$ - The position of UAV at each transition k .
6. $\gamma(k), k = 0, \dots, K$ - The time stamp at each transition k .
7. $\eta(k), k = 0, \dots, K$ - The energy of UAV at each transition k .

At each transition from a position ρ_i to ρ_j , the following constraint must be satisfied.

$$\begin{aligned} \rho(k) &= \rho_i \wedge \rho(k+1) = \rho_j \\ \implies (\gamma(k+1) &= \gamma(k) + \gamma_{ij} + \gamma_j), \\ \rho(k) &= \rho_i \wedge \rho(k+1) = \rho_j \\ \implies (\eta(k+1) &= \eta(k) + \eta_{ij} + \eta_j), \end{aligned} \quad (25)$$

where γ_{ij} is the transition time from ρ_i to ρ_j , γ_j is the slack time at ρ_j position, η_{ij} is the transition energy from ρ_i to ρ_j and η_j is the slack energy at ρ_j position. We evaluate γ_{ij} as follows.

$$\gamma_{ij} = \begin{cases} \gamma_d, & i = N_{q_i}^g \wedge j = N_{q_j}^g, \\ \phi_\gamma(i, j), & \text{else.} \end{cases} \quad (26)$$

Here, $\phi_\gamma(i, j)$ is a function that evaluates the transition time for any given pair of nodes when traveling at UAV velocity. The slack time, γ_j , is set as follows.

$$\gamma_j = \begin{cases} \gamma_d, & j = N_q^d, \\ 0, & \text{else.} \end{cases} \quad (27)$$

Then, we evaluate the transition energy η_{ij} as follows.

$$\eta_{ij} = \begin{cases} B_{\text{charge}}^a, & i = N_{q_i}^g \wedge j = N_{q_j}^g, \\ \psi_\eta(i, j), & \text{else.} \end{cases} \quad (28)$$

Here, $\psi_\eta(i, j)$ is a function that evaluates the transition energy consumed between any given pair of nodes when traveling at UAV velocity. The slack energy, η_j is set as follows.

$$\eta_j = \begin{cases} B_{\text{charge}}^a, & j = N_r^d, \\ 0, & \text{else.} \end{cases} \quad (29)$$

The following constraint is added to ensure that the specification is satisfied.

$$\bigwedge_{\forall N^v} \left(\bigvee_{\forall \rho(k)} (\rho(k) = N^v) \right) = True \quad (30)$$

Further, any UGV charging location is set to be active at the corresponding time as follows.

$$\rho(k) = N_q^g \implies \gamma(k) = \gamma_q^g. \quad (31)$$

At each depot charging instance, we set the start time to be an integer multiple of γ_d .

$$\bigwedge_{\forall N_r^d} \left(\bigvee_{\forall i \in \{1, \dots, K\}} (\rho(k) = N_r^d \implies \gamma(k) = i\gamma_d) \right) = True. \quad (32)$$

The AO problem is formulated using the previously defined constraints for each vehicle. Any feasible solution τ that will satisfy the set of constraints satisfies the mission. An SMT solver, Z3, is used to generate a feasible solution. However, the solution improvement is imposed as follows in Algorithm 1.

$$f_s(\zeta_{d_1}) \geq f_s(\zeta_{d_2}). \quad (33)$$

AO solution improves the plan from TDO for each UAV agent while satisfying the task site assignments. Above formulation allows both TDO and AO to be used as solvers in the proposed iterative framework in Algorithm 1. With sampled systems defined, we set $\Omega_1 = \text{TDO}$ and $\Omega_2 = \text{AO}$. This is because TDO can compute feasible solutions quickly, while AO can reduce the sub-optimality while exploiting computation resources. Further, TDO and AO are implemented in the shrinking horizon execution scheme described in Algorithm 2, which improves the plan further in between execution steps. In the next section, we analyze the results of this implementation of the proposed solution framework for planning tasks in the running example.

VI. IMPLEMENTATION RESULTS

A. Direct Implementation Vs. Proposed Implementation

This section presents the results of implementing Algorithm 1 and Algorithm 2 for a multi-vehicle planning scenario, using the two aforementioned solvers, TDO and AO. The task is to compute a plan for the road monitoring task described in the previous section. We use a team of two UAVs and one UGV. The discretization time γ_d is chosen as 5 min. Therefore, the shrinking horizon execution step is 5-min long. We also set the timeouts of the solver such that the execution step is not violated. The planning algorithm was executed on an AMD Ryzen Threadripper processor at 3.9MHz with 64GB RAM. We compare the results of the following two approaches.

1. Open-loop execution of TDO (Direct Implementation).
A plan is computed using TDO and implemented for the whole planning horizon. Therefore, we do not employ a secondary solver for reducing sub-optimality nor trigger any re-planning after the first computation.
2. Shrinking horizon execution of iterative planning (Proposed Implementation).

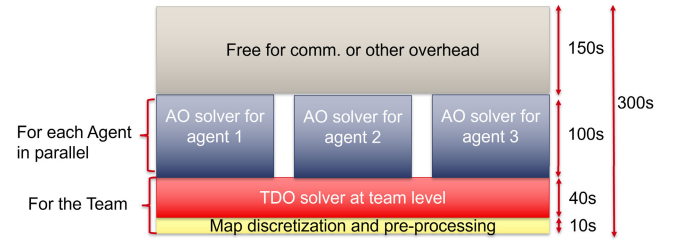


Fig. 9. Average compute time distribution: TDO represents Direct Implementation, while TDO+AO represents the proposed approach.

TABLE II
RESULT COMPARISON BETWEEN DIRECT IMPLEMENTATION AND PROPOSED IMPLEMENTATION

Comparison	Direct Implementation	Proposed Implementation
Plan time	120mins	105mins
Compute time	50s	150s

A plan is computed using Algorithm 1 employing TDO and AO iteratively at each step of execution through a shrinking horizon using algorithm 2. Therefore, two solvers are used iteratively in planning, and at each step of execution, re-planning is triggered.

Figure 9 shows the distribution of compute time for the proposed approach. We run TDO to compute a plan for the whole team for the complete horizon. If this plan is implemented, it corresponds to the Direct Implementation approach. However, using the proposed approach, we run Algorithm 1 with AO as the next step. As it is run at each agent individually, this enables running AO in parallel. For an execution step of 5 min, we finish planning through the proposed approach leaving a substantial amount of time for communications and other practical overheads. From the results of running the proposed approach, it requires an extra 100 seconds of computing time on average than direct implementation, while still staying within the boundary of the 5-min execution step. However, by trading off execution time, the proposed approach gains an improvement of the total mission time.

We present the results of the mission time improvement and compute time in Table II. It is evident from the results that the proposed algorithm minimizes plan time and improves the cost on an average of 12% by trading off compute time up to 100s. Further, Figure 10 shows the average improvement of planning time δ_f across each execution step for 25 instances of planning solutions generated by the proposed approach. If ζ_{TDO} and ζ_{AO} are implementations of plans generated from TDO and AO, respectively, we define,

$$\delta_f = \frac{f_s(\zeta_{TDO}) - f_s(\zeta_{AO})}{f_s(\zeta_{TDO})}. \quad (34)$$

We observe that the iterative planning framework and the shrinking horizon execution method contribute to continual improvements in the total mission time at each time step of execution.

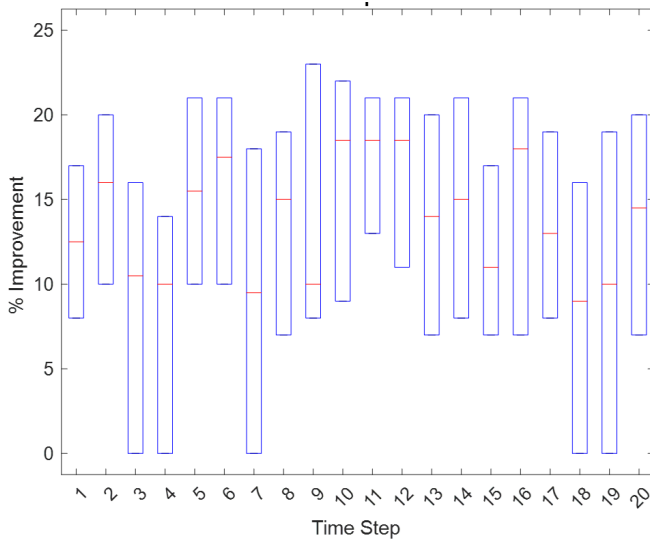


Fig. 10. Average planning time improvement δ_γ : The plot shows the mean and the interquartile range of plan time improvement at each execution step, averaged for 25 planning instances.

TABLE III
PROBLEM PARAMETERS IN COMPARED CASE STUDY

Parameters	GA	BO	PI
UGV	1	1	1
UAVs	2	2	2
Scenario	Figure 1	Figure 1	Figure 1
Depots	3	3	3
Discrete states	41	41	38
Energy dynamics	Eqs. (6), (5), (4)	Eqs. (6), (5), (4)	Eqs. (6), (5), (4)

B. Comparison: Proposed Implementation Vs. Genetic Algorithm and Bayesian Optimization Solvers

1) *Genetic Algorithm and Bayesian Optimization Solvers*: This section presents the results of the proposed implementation (PI) compared with a solver based on a genetic algorithm (GA) and a solver based on Bayesian optimization (BO) for the multi-vehicle planning scenario as presented in [33]. We selected these specific baselines as [33] uses the same energy-aware cooperative routing scenario described in Section V-A as the case study to evaluate the proposed solutions. Further, the approach taken in [33] follows the state of the art, which converts the original continuous state space into a hybrid state space. It also uses a hierarchical approach, where an initial step of clustering of task sites simplifies the problem for the next optimization step. The utilized problem parameters are detailed in the table III.

2) *Result Comparison*: The team composition was two UAVs and one UGV. The planning algorithms, PI and GA, were executed on an AMD Ryzen Threadripper processor at 3.9MHz with 64GB RAM while BO was executed on a server with similar computing power. We compare the results of the two approaches BO and GA against the proposed implementation. The detailed implementations of the GA-based solver

TABLE IV
RESULT COMPARISON BETWEEN GENETIC ALGORITHM (GA), BAYESIAN OPTIMIZATION (BO) AND PROPOSED IMPLEMENTATION (PI)

Metric	GA	BO	PI
Plan time	225min	249min	105min
Compute time	94min	15min	2.5min

and BO-based solver are discussed in [33]. The tested scenario was to complete the road monitoring task for the road network in Figure 7. The results of this implementation are presented in Table IV. We observe that the proposed approach achieves faster compute time while achieving more than 50% reduction of the total mission time. Due to the fast solution computation, it is evident that the proposed shrinking horizon execution scheme can be implemented in a real-time planning scenario.

C. Comparison: Proposed Implementation Vs. Optimization Solver

In this section, we compare the case study results of the proposed approach against optimal solutions from a Mixed Integer Programming (MIP) solver. One of the key advantages of our approach compared to optimization solvers is that we have an anytime property while optimization solvers, especially in real-life larger-scale NP-hard problems, often fail to deliver an optimal solution in a compute time that enables real-time implementation. The following results further validate the strengths of the proposed approach. We employ an MIP solver (Google ORtools) that solves the optimization problem (35), which minimizes the total plan times subject to the system and task constraints in Eqs. (10) - (24).

$$\begin{aligned} & \underset{\tau \in M}{\text{minimize}} \quad f_S(\zeta_\tau), \\ & \text{subject to: Eqs. (10) - (24).} \end{aligned} \quad (35)$$

The results of MIP solver, which is an optimal solver (OP) and Proposed Implementation (PI) is compared to evaluate the sensitivity to the number of agents and the number of task sites. We use the compute time and the sub-optimality as the evaluation metrics.

1) *No. Of Agents*: We adjust the number of agents by setting the number of groups of agents. A group consists of a UGV and two UAVs. We adjust the number of groups from 2 – 12, which changes the number of agents from 6 – 36. The number of task sites is held lower at 5. We set an upper bound of compute time as $T_d = 300s$, as that is the execution time step. It is considered a failed attempt if MIP or PI fails to deliver a solution within T_d .

We present the results of this case study in Figure 11. We see that the proposed approach generates feasible solutions faster than the optimal solution, as expected. While the compute time increases with the no. of agents, it is still within T_d . Thus, a feasible task-satisfying solution can always be generated. This is because when the number of agents increases, the feasible solution space also increases in size. Thus, finding a feasible solution is easier and faster, while searching for the optimal solution is computationally intensive.

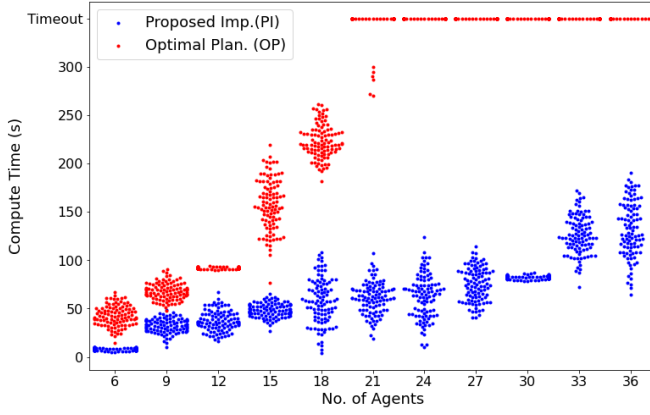


Fig. 11. Swarm plot of compute time distribution OP and PI for varying the no. of agents. PI - Each run generates a feasible solution. OP - Each run generates the optimal solution. Timed out runs are shown at the top.

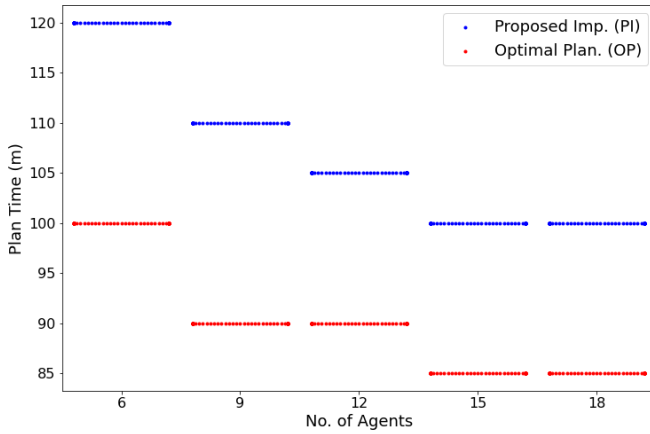


Fig. 12. Plan time distribution OP and PI for varying the no. of agents. PI - Each run generates a feasible solution. OP - Each run generates the optimal solution. Maximum sub-optimality is 22.2%.

The plan time for completed runs is shown in Figure 12, which excludes the timed-out runs. The PI solutions were sub-optimal, taking a maximum of 22.2% additional time than optimal solutions. The optimality gap decreases with increasing agents. This is because the agent-level optimization of iterative planning works best to improve the optimality gap when more variables are present at a higher number of agents.

2) *No. Of Task Sites*: We adjust the number of task sites by adjusting the goal states for the monitoring task from 3 – 20. The number of agents is held at 9. We set an upper bound of compute time as $T_d = 300s$, as that is the execution time step. It is considered a failed attempt if MIP or PI fails to deliver a solution within T_d .

We present the results of this case study in Figure 13. We see that the proposed approach generates feasible solutions faster than the optimal solution, as expected. While the compute time increases with the number of task sites, it is still within T_d . Thus, a feasible task-satisfying solution can always be generated. This is because when the number of task sites increases, the number of constraints that need to be satisfied increases. Therefore, searching for a satisfying solution becomes computationally harder, while searching for the optimal solution is further computationally expensive.

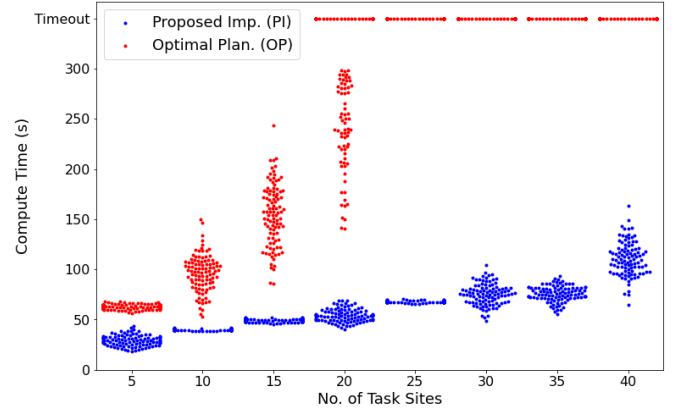


Fig. 13. Swarm plot of compute time distribution OP and PI for varying the number of task sites. PI - Each run generates a feasible solution. OP - Each run generates the optimal solution. Timed out runs are shown at the top.

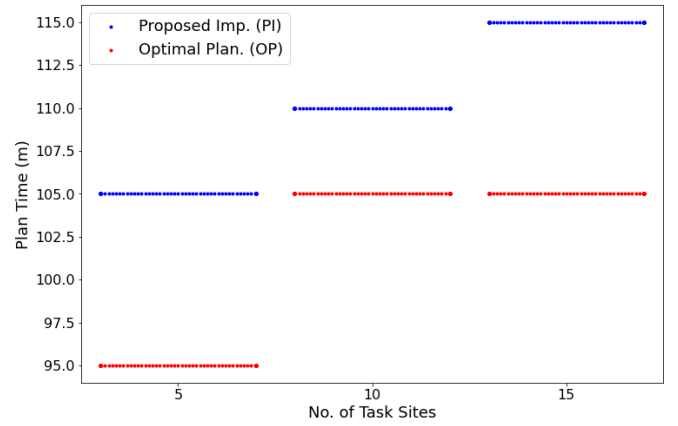


Fig. 14. Plan time distribution OP and PI for varying the no. of task sites. PI - Each run generates a feasible solution. OP - Each run generates the optimal solution. Maximum sub-optimality is 10.5%.

The plan time for completed runs is shown in Figure 14, which excludes the timed-out runs. On average, the solutions from PI were sub-optimal at 10.5% additional time than optimal solutions.

VII. CONCLUSION

In this paper, we present an iterative planning framework for multi-agent systems. We formulate a mathematical framework to represent multi-dimensional systems and planning tasks using transition systems. Solving planning problems of this nature is computationally complex. Therefore, we propose the idea of using sampled transition systems to discretize the problem to employ faster solvers. We propose an iterative approach that defines a solution framework using multiple solvers to continually improve a plan using a given amount of computation time. The proposed approach ensures that a solution is always available for execution, while the iterative process continues on further optimization. We also propose a shrinking horizon execution of the plan to enable re-planning at each step of execution for continual improvement of the plan. Further, we also derive and prove the necessary conditions that guarantee the recursive feasibility of the proposed approach. The proposed theoretical framework has the ability

to be applied to plan for any generalized task site assignment using multiple solvers iteratively.

We validate the proposed approach using an energy-aware UAV-UGV cooperative route planning problem to satisfy a given task site assignment. The example presented in this paper validates the proposed framework by producing plans that satisfy the goal, using computing resources efficiently to preserve real-time implementation ability. The iterative planning framework consisting of two SMT solvers ensures the continual improvement of the plan when executed in a shrinking horizon.

In conclusion, the theoretical framework introduced in this paper to represent complex dynamical systems presents several avenues for future improvements. The proposed framework can be further extended to define planning tasks that go beyond the generalized task site assignments. Further, exploring methods to integrate uncertainty present in plan execution and communication into the theoretical framework is important. We believe the proposed framework has the potential to serve as a useful tool for researchers and practitioners working on multi-agent system planning problems.

REFERENCES

- [1] O. Thakoor, J. Garg, and R. Nagi, "Multiagent UAV routing: A game theory analysis with tight price of anarchy bounds," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 100–116, Jan. 2020.
- [2] J. G. Carlsson and S. Song, "Coordinated logistics with a truck and a drone," *Manage. Sci.*, vol. 64, no. 9, pp. 4052–4069, Sep. 2018.
- [3] V. Yordanova, H. Griffiths, and S. Hailes, "Rendezvous planning for multiple autonomous underwater vehicles using a Markov decision process," *IET Radar, Sonar Navigat.*, vol. 11, no. 12, pp. 1762–1769, Dec. 2017.
- [4] J. Xin, C. Meng, A. D. Ariano, F. Schulte, J. Peng, and R. R. Negenborn, "Energy-efficient routing of a multirobot station: A flexible time-space network approach," *IEEE Trans. Autom. Sci. Eng.*, pp. 1–15, 2022.
- [5] R. M. A. Swaszek and C. G. Cassandras, "Receding horizon control for station inventory management in a bike-sharing system," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 407–417, Jan. 2020.
- [6] Z. Fang and J. Mao, "Energy-efficient elevating transfer vehicle routing for automated multi-level material handling systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 3, pp. 1107–1123, Jul. 2020.
- [7] S. G. Manyam, K. Sundar, and D. W. Casbeer, "Cooperative routing for an air-ground vehicle team—Exact algorithm, transformation method, and heuristics," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 537–547, Jan. 2020.
- [8] S. Ramasamy, J. F. Reddinger, J. M. Dotterweich, M. A. Childers, and P. A. Bhounsule, "Cooperative route planning of multiple fuel-constrained unmanned aerial vehicles with recharging on an unmanned ground vehicle," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2021, pp. 155–164.
- [9] S. Ramasamy, J.-P.-F. Reddinger, J. M. Dotterweich, M. A. Childers, and P. A. Bhounsule, "Coordinated route planning of multiple fuel-constrained unmanned aerial systems with recharging on an unmanned ground vehicle for mission coverage," *J. Intell. Robot. Syst.*, vol. 106, no. 1, pp. 1–18, Sep. 2022.
- [10] M. Ostertag, N. Atanasov, and T. Rosing, "Trajectory planning and optimization for minimizing uncertainty in persistent monitoring applications," *J. Intell. Robot. Syst.*, vol. 106, no. 1, pp. 2.1–2.19, Sep. 2022.
- [11] J. Redding et al., "Distributed multi-agent persistent surveillance and tracking with health management," in *Proc. AIAA Guid. Navigat. Control Conf.*, Portland, OR, USA, 2011, pp. 8–11.
- [12] C. A. A. Tatsch, *Route Planning for Long-Term Robotics Missions*. Morgantown, WV, USA: West Virginia University, 2020.
- [13] D. Mitchell, N. Chakraborty, K. Sycara, and N. Michael, "Multi-robot persistent coverage with stochastic task costs," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, Sep. 2015, pp. 3401–3406.
- [14] N. Mathew, S. L. Smith, and S. L. Waslander, "Planning paths for package delivery in heterogeneous multirobot teams," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 4, pp. 1298–1308, Oct. 2015.
- [15] K. Chour et al., "A reactive energy-aware rendezvous planning approach for multi-vehicle teams," in *Proc. IEEE 18th Int. Conf. Autom. Sci. Eng. (CASE)*, Aug. 2022, pp. 537–542.
- [16] K. Yu, A. K. Budhiraja, S. Buebel, and P. Tokekar, "Algorithms and experiments on routing of unmanned aerial vehicles with mobile recharging stations," *J. Field Robot.*, vol. 36, no. 3, pp. 602–616, May 2019.
- [17] L. Vu, D. M. Vu, M. H. Hà, and V. Nguyen, "The two-echelon routing problem with truck and drones," *Int. Trans. Oper. Res.*, vol. 29, no. 5, pp. 2968–2994, Sep. 2022. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1111/itor.13052>
- [18] H. Zhou, H. Qin, C. Cheng, and L.-M. Rousseau, "An exact algorithm for the two-echelon vehicle routing problem with drones," *Transp. Res. B, Methodol.*, vol. 168, pp. 124–150, Feb. 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0191261523000024>
- [19] P. Maini, K. Sundar, M. Singh, S. Rathinam, and P. B. Sujit, "Cooperative aerial-ground vehicle route planning with fuel constraints for coverage applications," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 55, no. 6, pp. 3016–3028, Dec. 2019.
- [20] M. Kenzin, I. Bychkov, and N. Maksimkin, "Coordinated recharging of heterogeneous mobile robot teams during continuous large scale missions," in *Proc. 7th Int. Conf. Control, Decis. Inf. Technol. (CoDIT)*, vol. 1, Jun. 2020, pp. 745–750.
- [21] M. Rappaport and C. Bettstetter, "Coordinated recharging of mobile robots during exploration," in *Proc. IEEE Int. Conf. Intell. Robots Syst.*, Sep. 2017, pp. 6809–6816.
- [22] Z. Xing, W. Wu, H. Niu, and R. Hu, "Battery charge dispatching in multi-robot systems," in *Proc. IEEE Int. Conf. Netw., Sens. Control (ICNSC)*, vol. 1, Dec. 2021, pp. 1–6.
- [23] E. Stump and N. Michael, "Multi-robot persistent surveillance planning as a vehicle routing problem," in *Proc. IEEE Int. Conf. Autom. Sci. Eng.*, Aug. 2011, pp. 569–575.
- [24] K. Sundar, S. Venkatachalam, and S. Rathinam, "Formulations and algorithms for the multiple depot, fuel-constrained, multiple vehicle routing problem," in *Proc. Amer. Control Conf. (ACC)*, Jul. 2016, pp. 6489–6494.
- [25] X. Wen and G. Wu, "Heterogeneous multi-drone routing problem for parcel delivery," *Transp. Res. C, Emerg. Technol.*, vol. 141, Aug. 2022, Art. no. 103763. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0968090X22001966>
- [26] S. Teng et al., "Motion planning for autonomous driving: The state of the art and future perspectives," *IEEE Trans. Intell. Veh.*, vol. 8, no. 6, pp. 3692–3711, Jun. 2023.
- [27] L. Dong, Z. He, C. Song, and C. Sun, "A review of mobile robot motion planning methods: From classical motion planning workflows to reinforcement learning-based architectures," *J. Syst. Eng. Electron.*, vol. 34, no. 2, pp. 439–459, Apr. 2023.
- [28] S. Gu, G. Chen, L. Zhang, J. Hou, Y. Hu, and A. Knoll, "Constrained reinforcement learning for vehicle motion planning with topological reachability analysis," *Robotics*, vol. 11, no. 4, p. 81, Aug. 2022. [Online]. Available: <https://www.mdpi.com/2218-6581/11/4/81>
- [29] C. Belta, B. Yordanov, and E. Gol, *Formal Methods for Discrete-Time Dynamical Systems* (Studies in Systems, Decision and Control). Cham, Switzerland: Springer, 2017.
- [30] P. Tabuada, *Verification and Control of Hybrid Systems: A Symbolic Approach*, 1st ed. New York, NY, USA: Springer, 2009.
- [31] L. Zhen, J. Gao, Z. Tan, S. Wang, and R. Baldacci, "Branch-price-and-cut for trucks and drones cooperative delivery," *IIEE Trans.*, vol. 55, no. 3, pp. 271–287, Mar. 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:249440594>
- [32] M. Coindreau, O. Gallay, and N. Zufferey, "Parcel delivery cost minimization with time window constraints using trucks and drones," *Networks*, vol. 78, no. 4, pp. 400–420, Dec. 2021, doi: [10.1002/net.22019](https://doi.org/10.1002/net.22019).
- [33] S. Ramasamy et al., "Heterogenous vehicle routing: Comparing parameter tuning using genetic algorithm and Bayesian optimization," in *Proc. Int. Conf. Unmanned Aircr. Syst. (ICUAS)*, Jun. 2022, pp. 104–113.
- [34] S. W. Cone, "Casualty evacuation optimization in a conflicted environment," M.S. thesis, Nav. Postgraduate School, Monterey, CA, USA, 2022.
- [35] K. Marler, R. Yoshida, and C. Vogiatzis, "U.S. Marine Corps rapid planning and logistics routing against uncertainty," *Nav. Eng. J.*, vol. 135, no. 1, pp. 115–125, 2023. [Online]. Available: <https://www.ingentaconnect.com/content/asne/nej/2023/00000135/00000001/art00022>

- [36] N. Mathew, S. L. Smith, and S. L. Waslander, "A graph-based approach to multi-robot rendezvous for recharging in persistent tasks," in *Proc. IEEE Int. Conf. Robot. Autom.*, May 2013, pp. 3497–3502.
- [37] R. Zhang, Q. Zong, X. Zhang, L. Dou, and B. Tian, "Game of drones: Multi-UAV pursuit-evasion game with online motion planning by deep reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 10, pp. 7900–7909, Oct. 2023.
- [38] S. Zimmermann, G. Hakimifard, M. Zamora, R. Poranne, and S. Coros, "A multi-level optimization framework for simultaneous grasping and motion planning," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 2966–2972, Apr. 2020.
- [39] C. R. Garrett et al., "Integrated task and motion planning," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 4, pp. 265–293, May 2021, doi: [10.1146/annurev-control-091420-084139](https://doi.org/10.1146/annurev-control-091420-084139).
- [40] P. A. Abdulla, "Well (and better) quasi-ordered transition systems," *Bull. Symbolic Log.*, vol. 16, no. 4, pp. 457–515, Dec. 2010.
- [41] Y. Cao, G. Chen, and E. E. Kerre, "Bisimulations for fuzzy-transition systems," *IEEE Trans. Fuzzy Syst.*, vol. 19, no. 3, pp. 540–552, Jun. 2011.
- [42] Y. Cao, S. X. Sun, H. Wang, and G. Chen, "A behavioral distance for fuzzy-transition systems," *IEEE Trans. Fuzzy Syst.*, vol. 21, no. 4, pp. 735–747, Aug. 2013.
- [43] A. A. Julius and G. J. Pappas, "Approximations of stochastic hybrid systems," *IEEE Trans. Autom. Control*, vol. 54, no. 6, pp. 1193–1203, Jun. 2009.
- [44] A. Girard, A. A. Julius, and G. J. Pappas, "Approximate simulation relations for hybrid systems," *IFAC Proc. Volumes*, vol. 39, no. 5, pp. 106–111, 2006.
- [45] S. Coogan and M. Arcak, "Efficient finite abstraction of mixed monotone systems," in *Proc. 18th Int. Conf. Hybrid Syst., Comput. Control*. New York, NY, USA: Association for Computing Machinery, Apr. 2015, pp. 58–67, doi: [10.1145/2728606.2728607](https://doi.org/10.1145/2728606.2728607).
- [46] P. Naumov and J. Tao, "Coalition power in epistemic transition systems," in *Proc. 16th Conf. Auto. Agents MultiAgent Syst.* Richland, SC, USA: International Foundation for Autonomous Agents and Multiagent Systems, 2017, pp. 723–731.



Neelanga Thelasingha received the B.Sc. degree (Hons.) in electrical and electronic engineering from the University of Peradeniya, Sri Lanka, in 2017, and the M.S. degree in computer and systems engineering from the Rensselaer Polytechnic Institute, NY, USA, in 2021, where he is currently pursuing the Ph.D. degree with the Electrical Computer and Systems Engineering Department. His research interests include optimal control and motion planning in multi-agent autonomous systems.



A. Agung Julius (Senior Member, IEEE) received the Ph.D. degree in applied mathematics from the University of Twente, Twente, The Netherlands, in 2005. From 2005 to 2008, he was a Post-Doctoral Researcher with the University of Pennsylvania. Since 2008, he has been with the Department of Electrical, Computer, and Systems Engineering, Rensselaer Polytechnic Institute, Troy, NY, USA, where he is currently a Professor. His research interests include systems and control, systems biology, stochastic models in systems biology, control of biological systems, hybrid systems, and mathematical systems theory. He received the National Science Foundation CAREER Award in 2010.



James Humann received the B.S. degree in mechanical engineering from The University of Oklahoma and the M.S. and Ph.D. degrees in mechanical engineering from the University of Southern California. He has been with the DEVCOM Army Research Laboratory since 2017. He is currently a Mechanical Engineer with the Vehicle Applied Research Branch. His research interests include design, path planning, and simulation for multi-robot systems.



Jean-Paul Reddinger is currently an Aerospace Research Engineer with the DEVCOM Army Research Laboratory. He is also works on unmanned aerial systems (UAS) flight dynamics for modeling and simulation as part of the Vehicle Applied Research Branch. His background is in aeromechanics and controls for high-speed compound rotorcraft.



James Dotterweich received the bachelor's degree in aerospace engineering from The University of Colorado at Boulder, CO, USA, in 2008, and the Master of Science degree in mechanical engineering, with a focus on robotics from The University of Utah, Salt Lake City, UT, USA, in 2015. He is currently a Robotics Research Engineer with the DEVCOM Army Research Laboratory, Aberdeen, MD, USA. He has been working on robotic systems and software development for unmanned aerial and ground vehicles for ARL for the past nine years.



Marshal Childers received the B.S. degree (Hons.) in mechanical engineering and the M.S. degree in mechanical engineering from the University of Maryland, Baltimore County (UMBC), in 2000 and 2001, respectively. He is currently a Mechanical Engineer with the DEVCOM Army Research Laboratory. He leads efforts in the integration, prototyping, and experimentation of autonomous robotics technologies and has collaborated on numerous experiments that were conducted to evaluate technologies developed by ARL-funded research programs. He is also an established Engineer with over 20 years of experience in the areas of autonomous technology assessment, integration, and mechanical design.